



ugr

Universidad
de Granada

PRÁCTICA 5

EXPERIMENTACIÓN CON EL SISTEMA DE SALIDA DE SONIDO

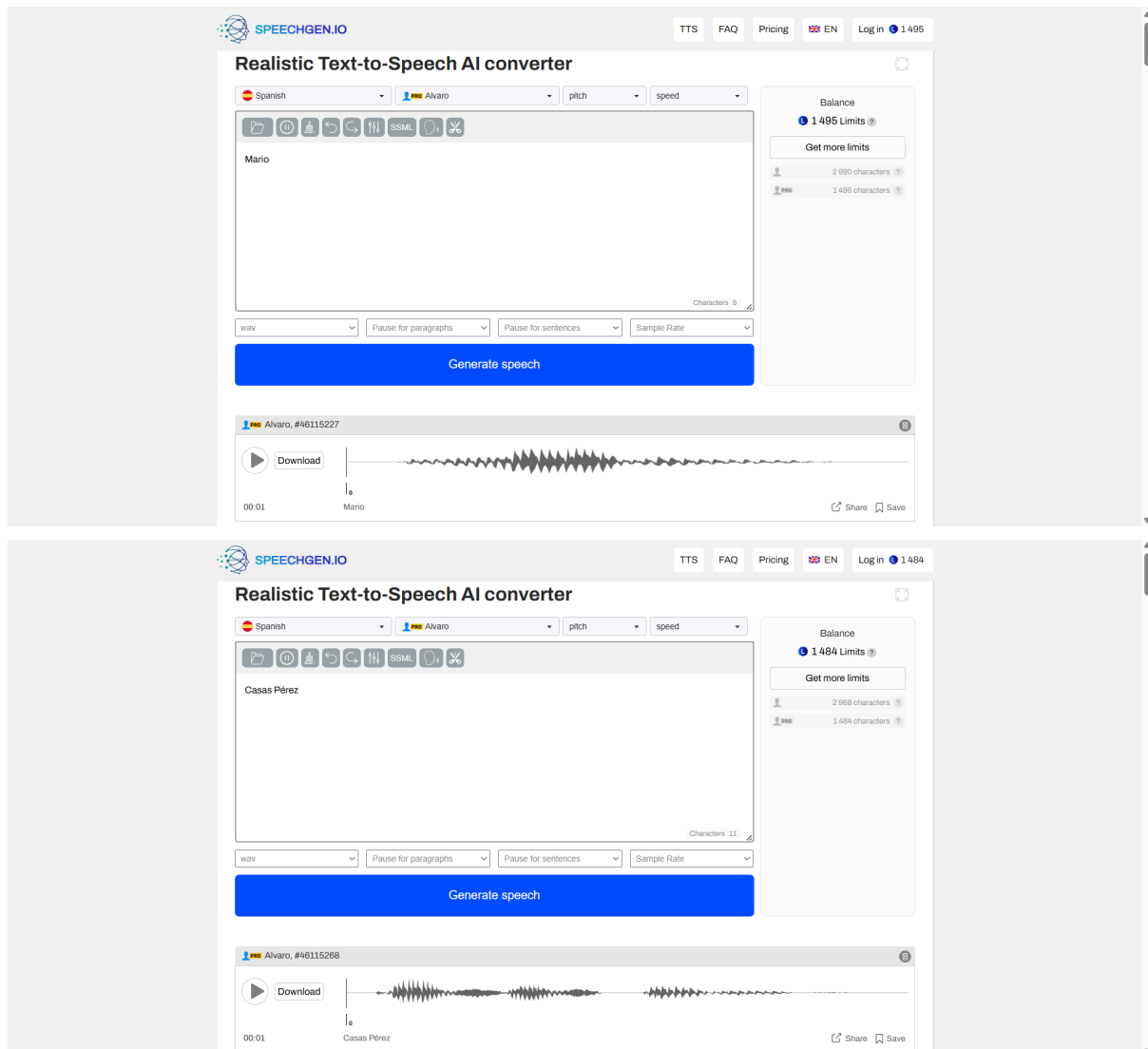
Periféricos y Dispositivos de Interfaz Humana

Autor:
Mario Casas Pérez

1. Ejercicio 1 mínimo	3
2. Ejercicio 2 mínimo	4
3. Ejercicio 3 mínimo	6
4. Ejercicio 4 mínimo	7
5. Ejercicio 5 mínimo	8
6. Ejercicio 6 mínimo	9
7. Ejercicio 1 opcional	10
8. Ejercicio 2 opcional	12
9. Ejercicio adicionales	14
a. Eliminación del segundo apellido (ejercicio adicional 1)	14
b. Eliminación del primer apellido (ejercicio 2 adicional)	16
c. Información y ondas del nombre con solo el primer apellido y con solo el segundo apellido (ejercicio 3).	18
d. Añadir silencio al final (ejercicio adicional 4).	20
e. Aplicación de filtro de amplitud (ejercicio adicional 5).	22
f. Eliminar porción de sonido con deletew (ejercicio adicional 6)	26
g. Cambiar primer apellido por segundo (ejercicio adicional 7)	28
h. Aumentar el tono y la velocidad modificando la frecuencia de muestreo (ejercicio adicional 8)	30
i. Reordenación de segmentos (ejercicio adicional 9)	32
j. Efecto de eco con retraso atenuado (ejercicio adicional 10)	34
k. Efecto de chorus (ejercicio adicional 11)	36
10. Bibliografía	38

1.Ejercicio 1 mínimo

Para este ejercicio, se nos pide crear 2 ficheros WAV en donde el primero se escuche el nombre de la persona que realiza la práctica y en el segundo sus apellidos. Para ello, nos iremos a la página de [SPEECHGEN.IO](https://speechgen.io) y crearemos los dos ficheros WAV seleccionando la extensión mencionada y pinchando en *Generate speech*. Pondremos el idioma en español, insertamos nuestro nombre y una vez generado le damos a *Download* (al igual que con los apellidos).



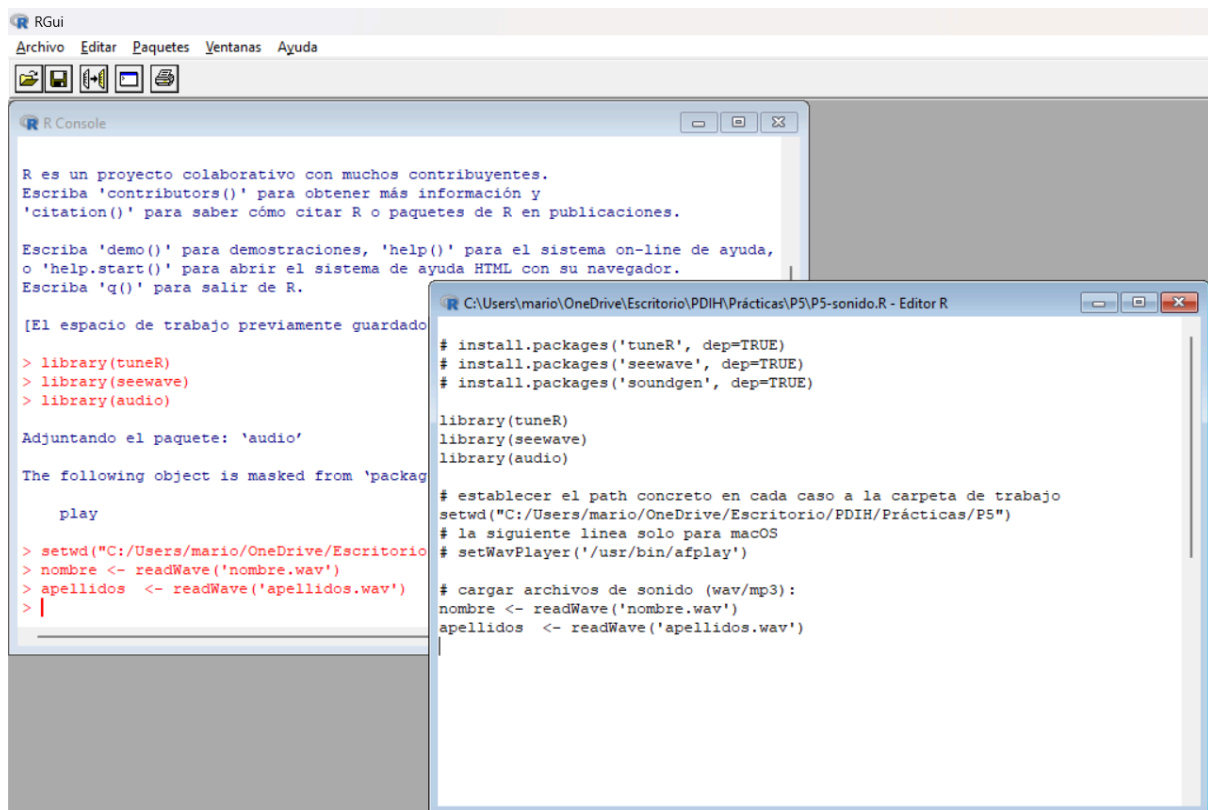
Una vez descargados ambos ficheros, lo pondremos en la misma carpeta donde vamos a hacer el script de la práctica para la realización de los siguientes ejercicios. Se renombraron estos ficheros como *nombre* y *apellidos*.

2. Ejercicio 2 mínimo

Para este ejercicio leeremos los dos ficheros creados anteriormente y dibujaremos la forma de onda de estos.

Para leerlo, usaremos el método *readWave*.

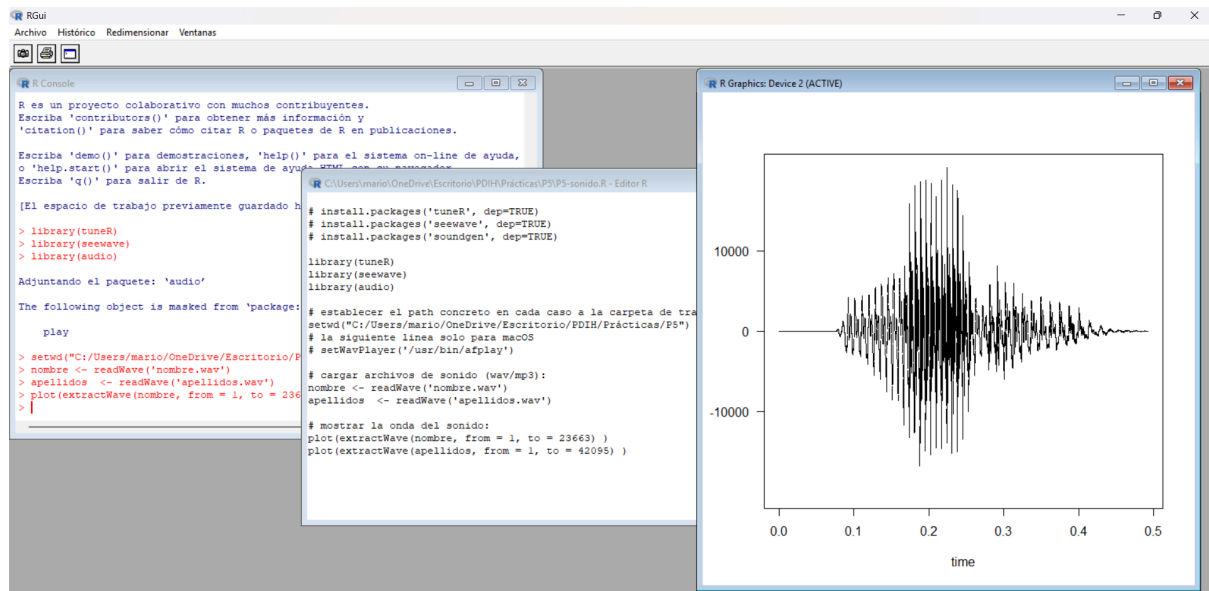
```
nombre <- readWave('nombre.wav')
apellidos <- readWave('apellidos.wav')
```



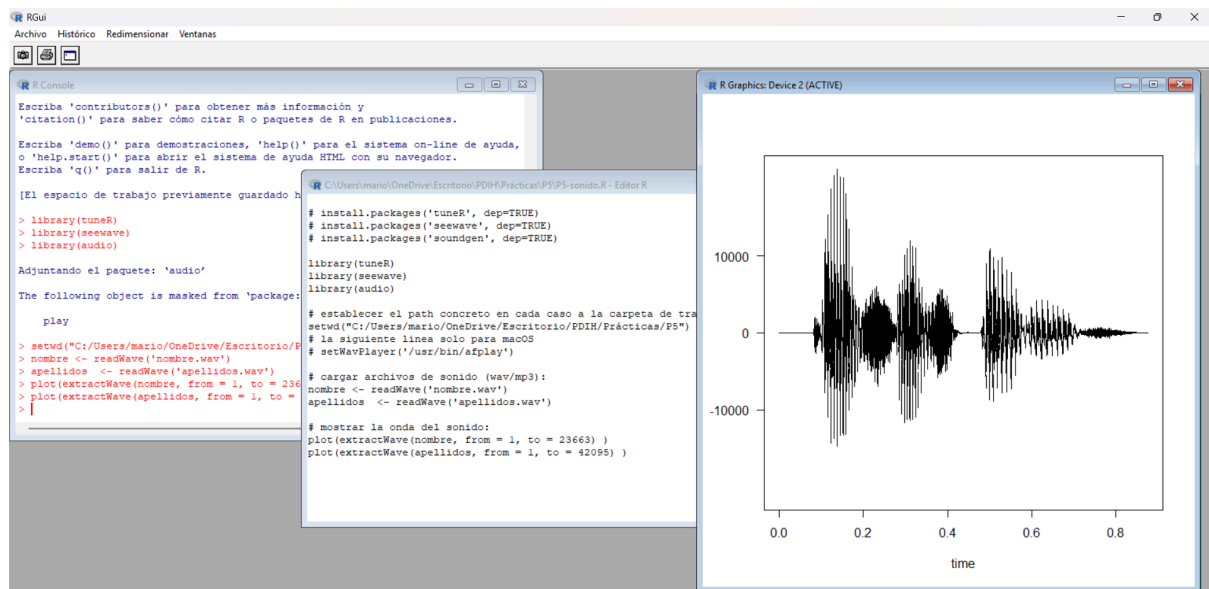
Para dibujar la onda de sonido, usaremos la función *plot*.

```
plot(extractWave(nombre, from = 1, to = 23663) )  
plot(extractWave(apellidos, from = 1, to = 42095) )
```

- nombre



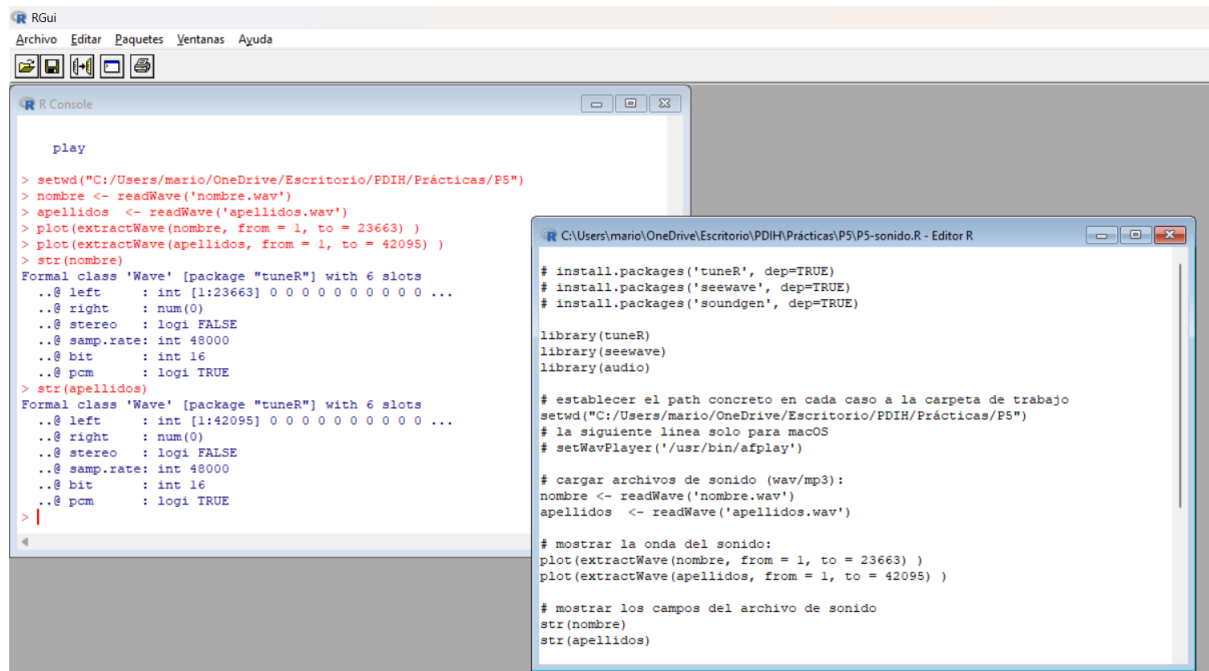
- apellidos



3.Ejercicio 3 mínimo

Para este ejercicio será necesario obtener la información de las cabeceras de ambos sonidos. Para ello, usaremos la función `str`.

```
str(nombre)
str(apellidos)
```



The screenshot shows the RGui interface. The R Console window on the left contains the following code and output:

```
play

> setwd("C:/Users/mario/OneDrive/Escritorio/PDIH/Prácticas/P5")
> nombre <- readWave('nombre.wav')
> apellidos <- readWave('apellidos.wav')
> plot(extractWave(nombre, from = 1, to = 23663) )
> plot(extractWave(apellidos, from = 1, to = 42095) )
> str(nombre)
Formal class 'Wave' [package "tuneR"] with 6 slots
..@ left      : int [1:23663] 0 0 0 0 0 0 0 0 0 0 ...
..@ right     : num(0)
..@ stereo    : logi FALSE
..@ samp.rate: int 48000
..@ bit       : int 16
..@ pcm       : logi TRUE
> str(apellidos)
Formal class 'Wave' [package "tuneR"] with 6 slots
..@ left      : int [1:42095] 0 0 0 0 0 0 0 0 0 0 ...
..@ right     : num(0)
..@ stereo    : logi FALSE
..@ samp.rate: int 48000
..@ bit       : int 16
..@ pcm       : logi TRUE
> |
```

The R Editor window on the right shows the following code:

```
# install.packages('tuneR', dep=TRUE)
# install.packages('seewave', dep=TRUE)
# install.packages('soundgen', dep=TRUE)

library(tuneR)
library(seewave)
library(audio)

# establecer el path concreto en cada caso a la carpeta de trabajo
setwd("C:/Users/mario/OneDrive/Escritorio/PDIH/Prácticas/P5")
# la siguiente linea solo para macOS
# setWavPlayer('/usr/bin/afplay')

# cargar archivos de sonido (wav/mp3):
nombre <- readWave('nombre.wav')
apellidos <- readWave('apellidos.wav')

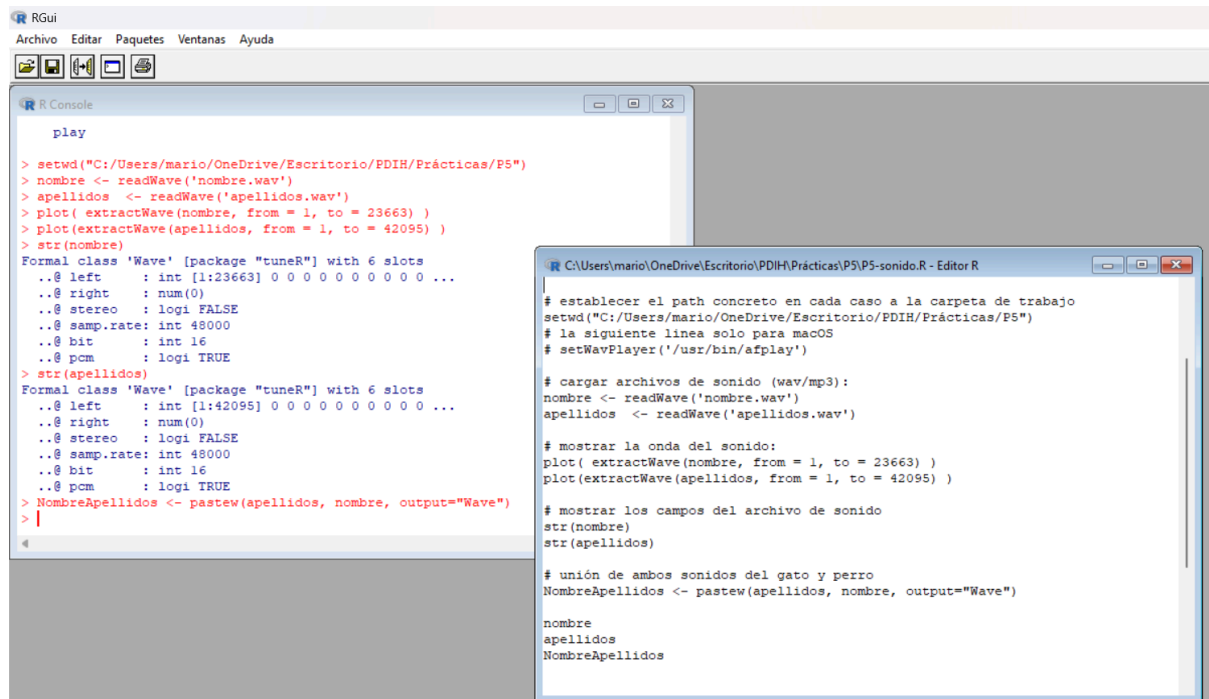
# mostrar la onda del sonido:
plot(extractWave(nombre, from = 1, to = 23663) )
plot(extractWave(apellidos, from = 1, to = 42095) )

# mostrar los campos del archivo de sonido
str(nombre)
str(apellidos)
```

4. Ejercicio 4 mínimo

En este ejercicio uniremos ambos sonidos para escuchar el nombre y los apellidos correctamente en un solo sonido. Para ello, usaremos la función *pastew*.

```
NombreApellidos <- pastew(apellidos, nombre, output="Wave")
```



Vamos a ver los datos del nombre, de los apellidos, y de la unión de ambos con el método mencionado previamente. Para ello, ejecutaremos lo siguiente.

```
nombre
apellidos
NombreApellidos
```

```
> NombreApellidos <- pastew(apellidos, nombre, output="Wave")
> nombre

Wave Object
  Number of Samples: 23663
  Duration (seconds): 0.49
  Samplingrate (Hertz): 48000
  Channels (Mono/Stereo): Mono
  PCM (integer format): TRUE
  Bit (8/16/24/32/64): 16

> apellidos

Wave Object
  Number of Samples: 42095
  Duration (seconds): 0.88
  Samplingrate (Hertz): 48000
  Channels (Mono/Stereo): Mono
  PCM (integer format): TRUE
  Bit (8/16/24/32/64): 16

> NombreApellidos

Wave Object
  Number of Samples: 65758
  Duration (seconds): 1.37
  Samplingrate (Hertz): 48000
  Channels (Mono/Stereo): Mono
  PCM (integer format): TRUE
  Bit (8/16/24/32/64): 16

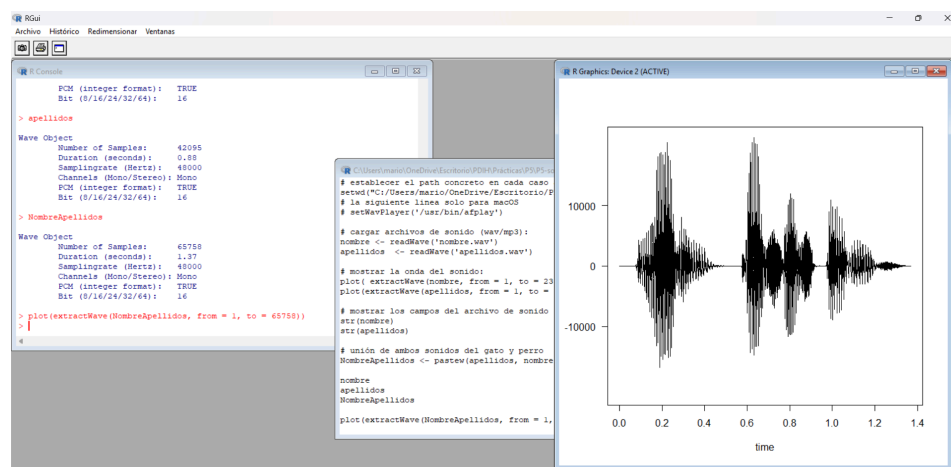
> |
```

5. Ejercicio 5 mínimo

Para este ejercicio dibujaremos la onda de la unión de ambos sonidos en NombreApellidos, así como la reproducción del sonido resultante.

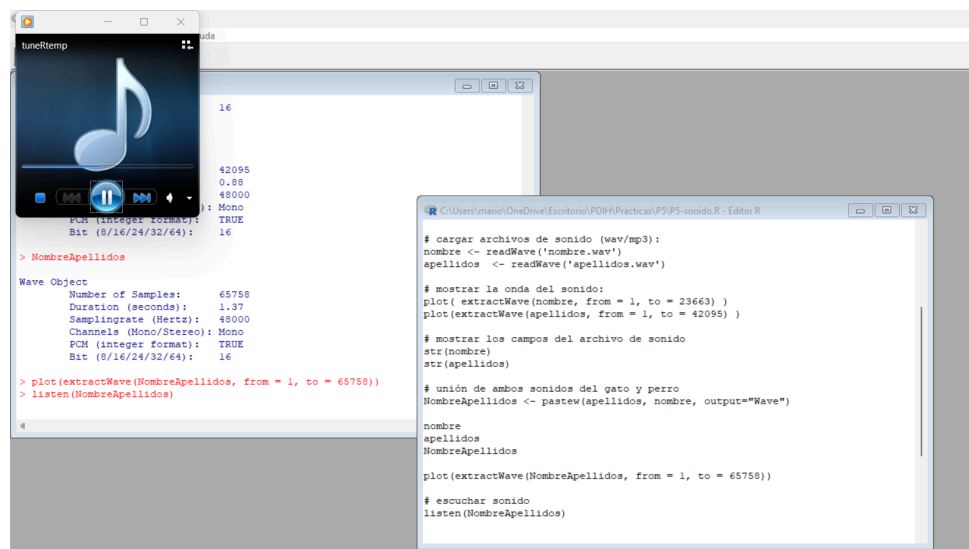
Para dibujar la onda con los dos sonidos (nuestro nombre y apellidos), usaremos el método *plot*.

```
plot(extractWave(NombreApellidos, from = 1, to = 65758))
```



Para reproducir el sonido resultante, usaremos el método *listen*.

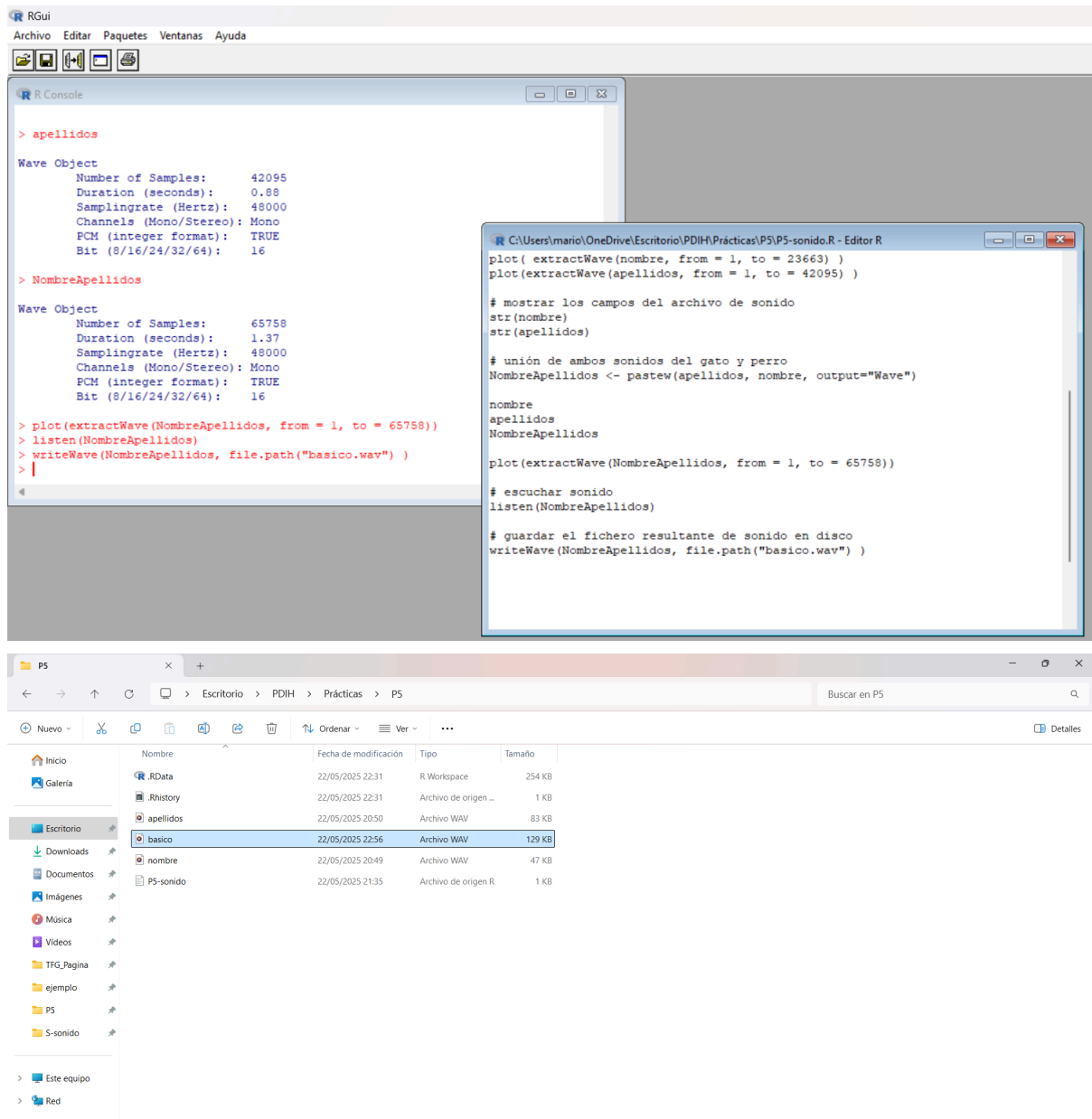
```
listen(NombreApellidos)
```



6. Ejercicio 6 mínimo

Para este ejercicio guardaremos el sonido de la unión de nuestro nombre y apellidos en un archivo nuevo llamado “basico.wav”. Por tanto, para almacenar el sonido obtenido en el formato que deseemos en disco, usaremos la función *writeWave*. Se guardará en la ruta que se especificó al principio del script (`setwd("ruta")`)

```
writeWave(NombreApellidos, file.path("basico.wav"))
```

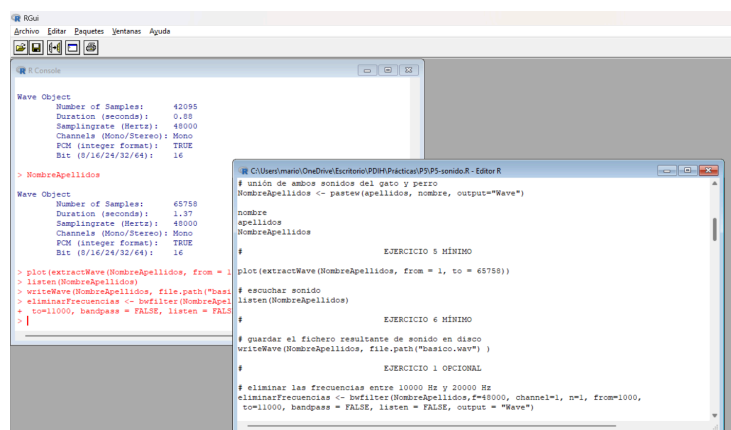


7. Ejercicio 1 opcional

Para este ejercicio pasaremos un filtro de frecuencia para eliminar las frecuencias entre 1.000 Hz y 11.000 Hz (se ha modificado el rango de frecuencias que se dice en el ejercicio dado a que se escucharía muy similar, por lo que se ha ajustado. No obstante, se haría igual para eliminar las frecuencias de 10.000 Hz a 20.000 Hz, lo único que habría que ajustar sería el *from* = 10.000 y *to* = 20.000). Posteriormente, almacenaremos la señal obtenida como un fichero WAV llamado "filtrado.wav".

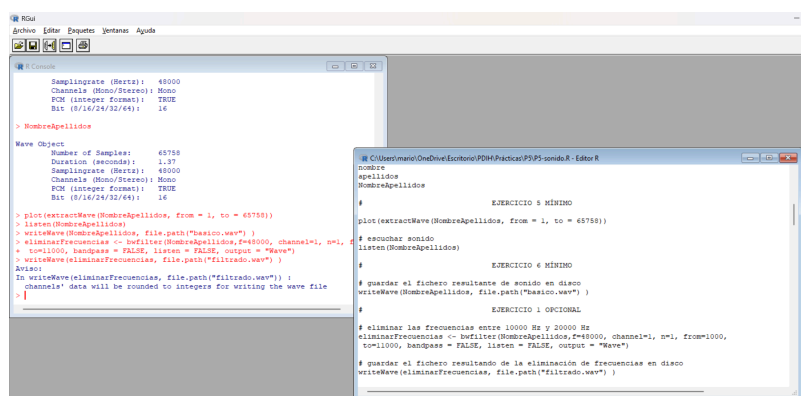
Para eliminar las frecuencias con los rangos descritos, usaremos la función *bwfilter*. Debemos de tener en cuenta que, para eliminar las frecuencias en estos rangos, se debe aplicar un filtro de rechazo de banda (bandpass = FALSE) para eliminar las frecuencias que se encuentran en dicho rango.

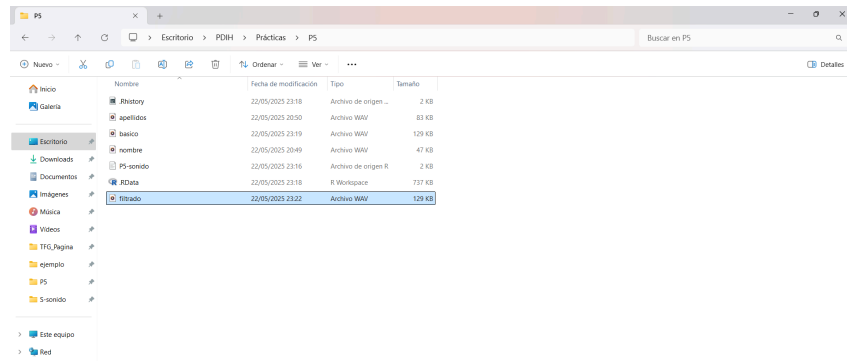
```
eliminarFrecuencias <- bwfilter(NombreApellidos,f=48000, channel=1, n=1,
from=1000, to=11000, bandpass = FALSE, listen = FALSE, output = "Wave")
```



Una vez ejecutado, pasamos a guardarlo en disco usando la función *writeWave*. Se guardará en la ruta que se especificó al principio del script (setwd("ruta"))

```
writeWave(eliminarFrecuencias, file.path("filtrado.wav"))
```



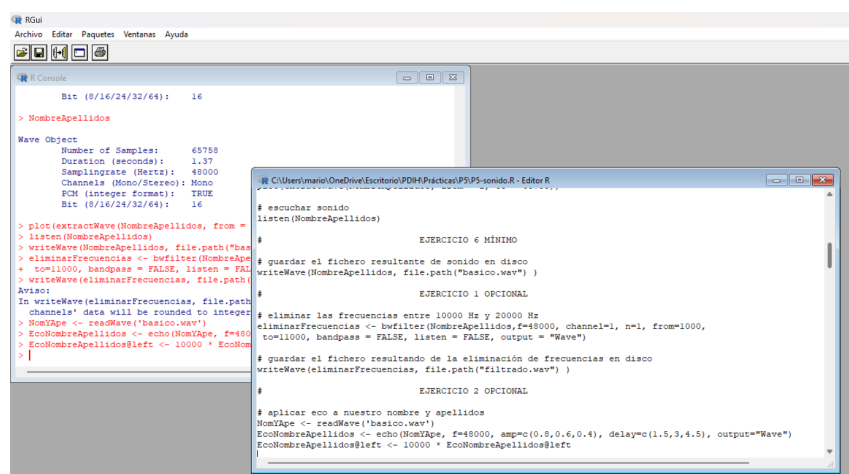


8. Ejercicio 2 opcional

En este ejercicio cogeremos el sonido guardado en “basico.wav” para aplicarle el efecto de eco. Guardaremos ese sonido en un archivo nuevo llamado “eco.wav”. Posteriormente, le daremos la vuelta al sonido y se guardará como un fichero llamado “alreves.wav”.

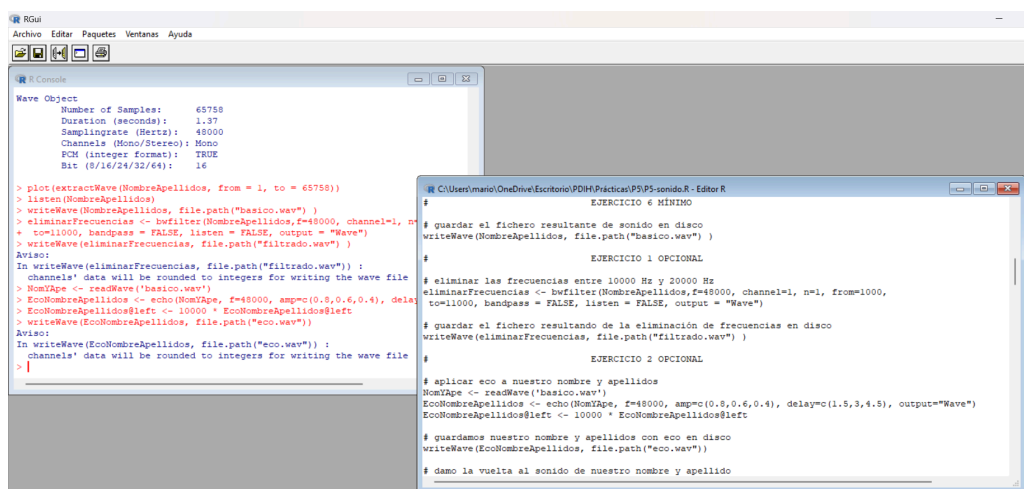
Lo primero de todo será aplicar el sonido de eco a nuestro fichero que tiene nuestro nombre y apellidos (*basico.wav*). La implementación del eco se hará con la función *echo*.

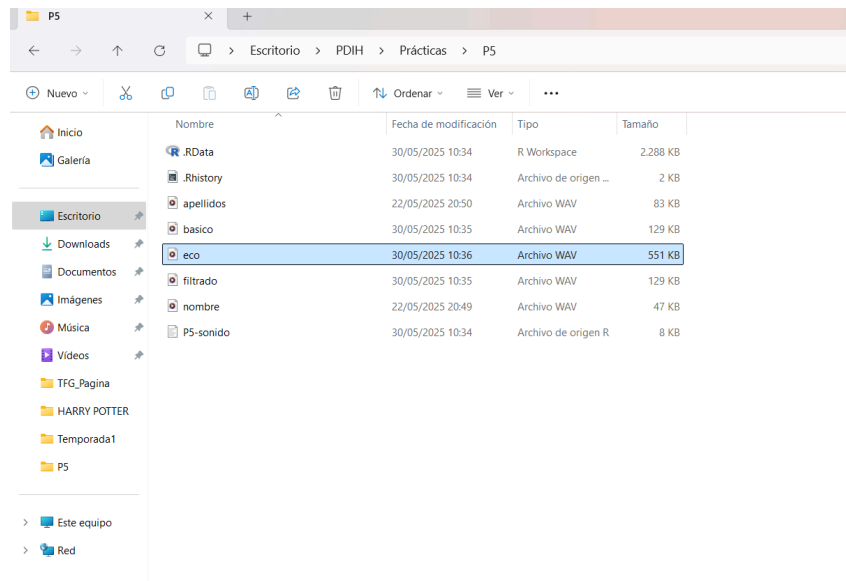
```
NomYApe <- readWave('basico.wav')
EcoNombreApellidos <- echo(NomYApe, f=48000, amp=c(0.8,0.6,0.4),
delay=c(1.5,3,4.5), output="Wave")
EcoNombreApellidos@left <- 10000 * EcoNombreApellidos@left
```



Ahora, pasamos a guardarlo en disco en un fichero llamado “eco.wav”. Esto lo haremos con la función *writeWave*.

```
writeWave(EcoNombreApellidos, file.path("eco.wav"))
```



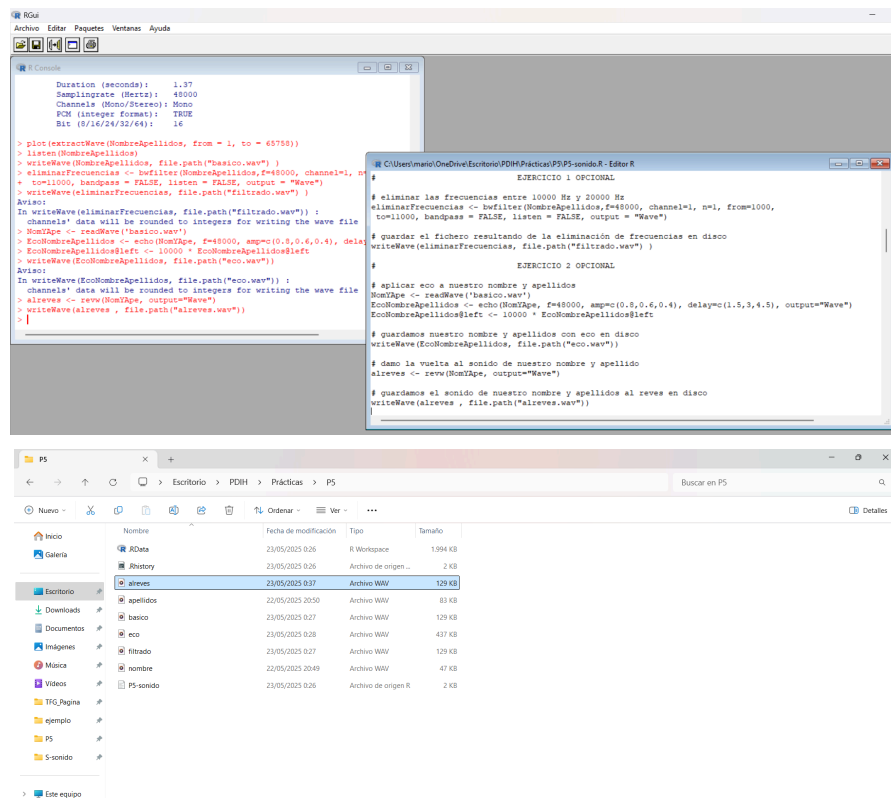


A continuación procedemos a darle la vuelta al sonido. Para ello, usaremos la función `revw`.

```
alreves <- revw(NomYape, output="Wave")
```

Finalmente, se procede a guardarlo en disco en un fichero llamado “alreves.wav”. Esto lo haremos con la función `writeWave`.

```
writeWave(alreves , file.path("alreves.wav"))
```

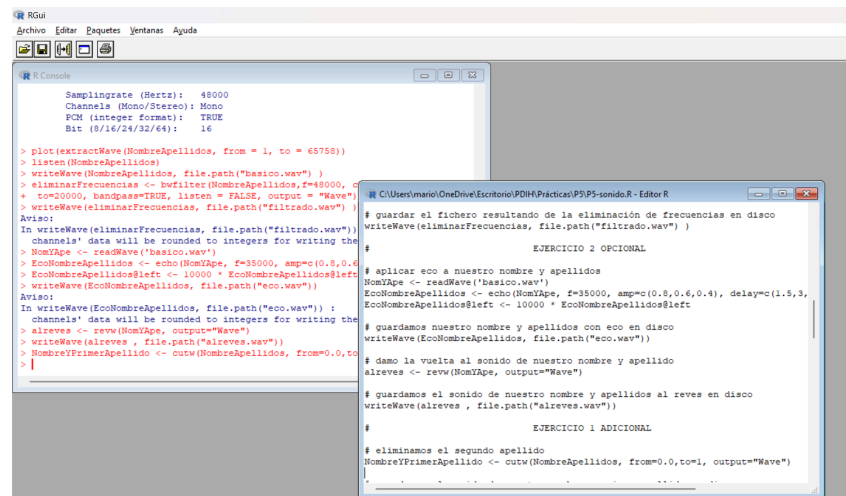


9. Ejercicio adicionales

a. Eliminación del segundo apellido (ejercicio adicional 1)

Para este ejercicio, vamos a eliminar nuestro segundo apellido (en mi caso Pérez) de la variable donde se almacena nuestro nombre completo (*NombreApellidos*). Para ello, usaremos la función *cutw*, dónde nos quedaremos con la porción de sonido que queremos (en mi caso, Mario Casas, que va desde 0 segundos a 1 segundos).

```
NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to=1, output="Wave")
```



```
RGui
Archivo  Editar  Paquetes  Ventanas  Ayuda

R Console
Samplingrate (Hertz): 48000
Channels (Mono/Stereo): Mono
PCM (integer format): TRUE
Bit (8/16/24/32/64): 16

> plot(extractWave(NombreApellidos, from = 1, to = 65758))
> listen(NombreApellidos)
> writeWave(NombreApellidos, file.path("basico.wav"))
> eliminarFrecuencias <- bwfilter(NombreApellidos, f=48000, c
+ to=20000, bandpass=TRUE, listen = FALSE, output = "Wave")
> writeWave(eliminarFrecuencias, file.path("filtrado.wav"))
Aviso:
In writeWave(eliminarFrecuencias, file.path("filtrado.wav")) :
channels' data will be rounded to integers for writing the
> NomYApe <- readWave("basico.wav")
> EcoNombreApellidos <- echo(NomYApe, f=35000, amp=c(0.8,0.6
> EcoNombreApellidos$left <- 10000 * EcoNombreApellidos$left
> writeWave(EcoNombreApellidos, file.path("eco.wav"))
Aviso:
In writeWave(EcoNombreApellidos, file.path("eco.wav")) :
channels' data will be rounded to integers for writing the
> alreves <- rev(NomYApe, output="Wave")
> writeWave(alreves, file.path("alreves.wav"))
> NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to
> |

C:\Users\mario\OneDrive\Escritorio\PDH\Prácticas\PS\PS-sonido.R - Editor R
# guardar el fichero resultando de la eliminación de frecuencias en disco
writeWave(eliminarFrecuencias, file.path("filtrado.wav"))

# EJERCICIO 2 OPCIONAL
# aplicar eco a nuestro nombre y apellidos
NomYApe <- readWave("basico.wav")
EcoNombreApellidos <- echo(NomYApe, f=35000, amp=c(0.8,0.6,0.4), delay=c(1.5,3,
EcoNombreApellidos$left <- 10000 * EcoNombreApellidos$left

# guardamos nuestro nombre y apellidos con eco en disco
writeWave(EcoNombreApellidos, file.path("eco.wav"))

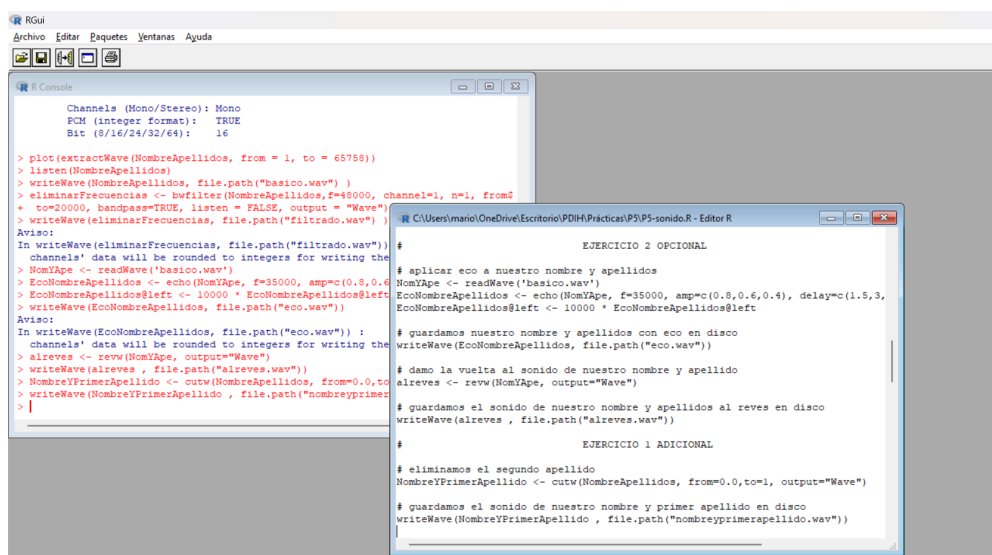
# damos la vuelta al sonido de nuestro nombre y apellido
alreves <- rev(NomYApe, output="Wave")

# guardamos el sonido de nuestro nombre y apellidos al reves en disco
writeWave(alreves, file.path("alreves.wav"))

# EJERCICIO 1 ADICIONAL
# eliminamos el segundo apellido
NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to=1, output="Wave")
# guardamos el sonido de nuestro nombre y primer apellido en disco
writeWave(NombreYPrimerApellido, file.path("nombreyprimerapellido.wav"))
```

Finalmente, lo guardamos en disco en un fichero llamado "nombreyprimerapellido.wav".

```
writeWave(NombreYPrimerApellido, file.path("nombreyprimerapellido.wav"))
```



```
RGui
Archivo  Editar  Paquetes  Ventanas  Ayuda

R Console
Channels (Mono/Stereo): Mono
PCM (integer format): TRUE
Bit (8/16/24/32/64): 16

> plot(extractWave(NombreApellidos, from = 1, to = 65758))
> listen(NombreApellidos)
> writeWave(NombreApellidos, file.path("basico.wav"))
> eliminarFrecuencias <- bwfilter(NombreApellidos, f=48000, channel=1, n=1, from$
+ to=20000, bandpass=TRUE, listen = FALSE, output = "Wave")
> writeWave(eliminarFrecuencias, file.path("filtrado.wav"))
Aviso:
In writeWave(eliminarFrecuencias, file.path("filtrado.wav")) :
channels' data will be rounded to integers for writing the
> NomYApe <- readWave("basico.wav")
> EcoNombreApellidos <- echo(NomYApe, f=35000, amp=c(0.8,0.6
> EcoNombreApellidos$left <- 10000 * EcoNombreApellidos$left
> writeWave(EcoNombreApellidos, file.path("eco.wav"))
Aviso:
In writeWave(EcoNombreApellidos, file.path("eco.wav")) :
channels' data will be rounded to integers for writing the
> alreves <- rev(NomYApe, output="Wave")
> writeWave(alreves, file.path("alreves.wav"))
> NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to
> writeWave(NombreYPrimerApellido, file.path("nombreyprimer
> |

C:\Users\mario\OneDrive\Escritorio\PDH\Prácticas\PS\PS-sonido.R - Editor R
# EJERCICIO 2 OPCIONAL
# aplicar eco a nuestro nombre y apellidos
NomYApe <- readWave("basico.wav")
EcoNombreApellidos <- echo(NomYApe, f=35000, amp=c(0.8,0.6,0.4), delay=c(1.5,3,
EcoNombreApellidos$left <- 10000 * EcoNombreApellidos$left

# guardamos nuestro nombre y apellidos con eco en disco
writeWave(EcoNombreApellidos, file.path("eco.wav"))

# damos la vuelta al sonido de nuestro nombre y apellido
alreves <- rev(NomYApe, output="Wave")

# guardamos el sonido de nuestro nombre y apellidos al reves en disco
writeWave(alreves, file.path("alreves.wav"))

# EJERCICIO 1 ADICIONAL
# eliminamos el segundo apellido
NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to=1, output="Wave")
# guardamos el sonido de nuestro nombre y primer apellido en disco
writeWave(NombreYPrimerApellido, file.path("nombreyprimerapellido.wav"))
```

P5

Escritorio > PDIH > Prácticas > P5

Nuevo

Ordenar

Ver

Inicio

Galería

Escritorio

Downloads

Documentos

Imágenes

Música

Videos

TFG_Pagina

ejemplo

P5

S-sonido

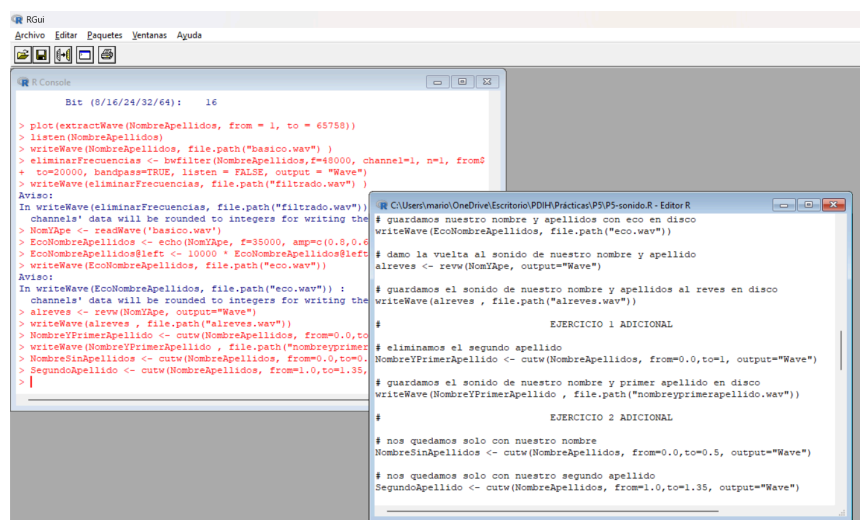
Este equipo

Nombre	Fecha de modificación	Tipo	Tamaño
.Rhistory	24/05/2025 10:20	Archivo de origen ...	2 KB
alreves	24/05/2025 10:21	Archivo WAV	129 KB
apellidos	22/05/2025 20:50	Archivo WAV	83 KB
basico	24/05/2025 10:21	Archivo WAV	129 KB
basicoConFiltro15	23/05/2025 22:02	Archivo WAV	129 KB
basicoConFiltro35	23/05/2025 22:02	Archivo WAV	129 KB
basicoConSilencio	23/05/2025 22:01	Archivo WAV	223 KB
deletePrimerApellido	23/05/2025 23:28	Archivo WAV	82 KB
eco	24/05/2025 10:21	Archivo WAV	437 KB
filtrado	24/05/2025 10:21	Archivo WAV	129 KB
nombre	22/05/2025 20:49	Archivo WAV	47 KB
nombreCambioPrimerSegundoApellido	23/05/2025 22:53	Archivo WAV	129 KB
P5-sonido	24/05/2025 10:20	Archivo de origen R	6 KB
.RData	24/05/2025 10:20	R Workspace	2.339 KB
nombreyprimerapellido	24/05/2025 10:22	Archivo WAV	94 KB

b. Eliminación del primer apellido (ejercicio 2 adicional)

Para este ejercicio vamos a eliminar nuestro primer apellido (en mi caso Casas) de la variable donde se almacena nuestro nombre completo (*NombreApellidos*). Para ello, usaremos la función *cutw*, dónde nos quedaremos primeramente con la porción de sonido donde se encuentra nuestro nombre, y luego nos quedaremos con la porción de sonido donde se escucha nuestro segundo apellido. Mirando el fichero de sonido, se ha podido observar que de 0 a 0.5 segundos es cuando se escucha mi nombre y desde 1 segundo hasta 1.35 segundo es cuando se dice mi apellido.

```
NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.5, output="Wave")
SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35, output="Wave")
```



```
Bit (8/16/24/32/64): 16

> plot(extractWave(NombreApellidos, from = 1, to = 65758))
> listen(NombreApellidos)
> writeWave(NombreApellidos, file.path("basico.wav"))
> eliminarFrecuencias <- hfilter(NombreApellidos, f=4000, channel=1, n=1, from2
+ to=20000, bandpass=TRUE, listen = FALSE, output = "Wave")
> writeWave(eliminarFrecuencias, file.path("filtrado.wav"))
Aviso:
In writeWave(eliminarFrecuencias, file.path("filtrado.wav")) :
channels' data will be rounded to integers for writing the
> NomYape <- readWave("basico.wav")
> EcoNombreApellidos <- echo(NomYape, f=35000, amp=0.8,0.4
> EcoNombreApellidos$left <- 10000 * EcoNombreApellidos$left
> writeWave(EcoNombreApellidos, file.path("eco.wav"))
Aviso:
In writeWave(EcoNombreApellidos, file.path("eco.wav")) :
channels' data will be rounded to integers for writing the
> alreves <- revw(NomYape, output="Wave")
> writeWave(alreves, file.path("alreves.wav"))
> NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to
> writeWave(NombreYPrimerApellido, file.path("nombreyprimer
> NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.
> SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35
> |

# guardamos nuestro nombre y apellidos con eco en disco
writeWave(EcoNombreApellidos, file.path("eco.wav"))

# damos la vuelta al sonido de nuestro nombre y apellido
alreves <- revw(NomYape, output="Wave")

# guardamos el sonido de nuestro nombre y apellidos al reves en disco
writeWave(alreves, file.path("alreves.wav"))

# EJERCICIO 1 ADICIONAL

# eliminamos el segundo apellido
NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to=1, output="Wave")

# guardamos el sonido de nuestro nombre y primer apellido en disco
writeWave(NombreYPrimerApellido, file.path("nombreyprimerapellido.wav"))

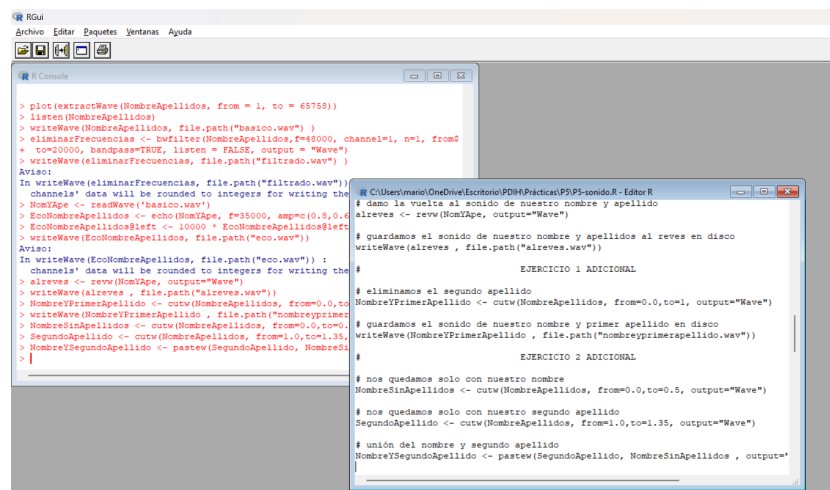
# EJERCICIO 2 ADICIONAL

# nos quedamos solo con nuestro nombre
NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.5, output="Wave")

# nos quedamos solo con nuestro segundo apellido
SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35, output="Wave")
```

Ahora, pasamos a unir ambos sonidos de tal manera que se escuche primeramente nuestro nombre seguido de nuestro segundo apellido. Para ello, usaremos la función *pastew*.

```
NombreYSegundoApellido <- pastew(SegundoApellido, NombreSinApellidos ,
output="Wave")
```



```
> plot(extractWave(NombreApellidos, from = 1, to = 65758))
> listen(NombreApellidos)
> writeWave(NombreApellidos, file.path("basico.wav"))
> eliminarFrecuencias <- hfilter(NombreApellidos, f=4000, channel=1, n=1, from2
+ to=20000, bandpass=TRUE, listen = FALSE, output = "Wave")
> writeWave(eliminarFrecuencias, file.path("filtrado.wav"))
Aviso:
In writeWave(eliminarFrecuencias, file.path("filtrado.wav")) :
channels' data will be rounded to integers for writing the
> NomYape <- readWave("basico.wav")
> EcoNombreApellidos <- echo(NomYape, f=35000, amp=0.8,0.4
> EcoNombreApellidos$left <- 10000 * EcoNombreApellidos$left
> writeWave(EcoNombreApellidos, file.path("eco.wav"))
Aviso:
In writeWave(EcoNombreApellidos, file.path("eco.wav")) :
channels' data will be rounded to integers for writing the
> alreves <- revw(NomYape, output="Wave")
> writeWave(alreves, file.path("alreves.wav"))
> NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to
> writeWave(NombreYPrimerApellido, file.path("nombreyprimer
> NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.
> SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35
> NombreYSegundoApellido <- pastew(SegundoApellido, NombreSinApellidos ,
> |

# damos la vuelta al sonido de nuestro nombre y apellido
alreves <- revw(NomYape, output="Wave")

# guardamos el sonido de nuestro nombre y apellidos al reves en disco
writeWave(alreves, file.path("alreves.wav"))

# EJERCICIO 1 ADICIONAL

# eliminamos el segundo apellido
NombreYPrimerApellido <- cutw(NombreApellidos, from=0.0,to=1, output="Wave")

# guardamos el sonido de nuestro nombre y primer apellido en disco
writeWave(NombreYPrimerApellido, file.path("nombreyprimerapellido.wav"))

# EJERCICIO 2 ADICIONAL

# nos quedamos solo con nuestro nombre
NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.5, output="Wave")

# nos quedamos solo con nuestro segundo apellido
SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35, output="Wave")

# unión del nombre y segundo apellido
NombreYSegundoApellido <- pastew(SegundoApellido, NombreSinApellidos , output="Wave")
```

Finalmente, guardamos un fichero de sonido conteniendo nuestro nombre y segundo apellido en disco usando la función `writeWave`. Este se llamará "nombreaysegundopellido.wav".

```
writeWave(NombreYSegundoApellido , file.path("nombreaysegundopellido.wav"))
```

```

> plot(extractWave(NombreApellidos, from = 1, to = 65758))
> listen(NombreApellidos)
> writeWave(NombreApellidos, file.path("basico.wav"))
> eliminarFrecuencias <- bwfilter(NombreApellidos, f=4000, channel=1, n=1, from$
+ to=20000, bandpass=TRUE, listen = FALSE, output = "Wave")
> writeWave(eliminarFrecuencias, file.path("filtrado.wav"))
Aviso:
In writeWave(eliminarFrecuencias, file.path("filtrado.wav")) :
channels' data will be rounded to integers for writing the
> NomApe <- readWave("basico.wav")
> EcoNombreApellidos <- echo(NomApe, f=35000, amp=c(0.8,0.6))
> EcoNombreApellidos$left <- 10000 * EcoNombreApellidos$left
> writeWave(EcoNombreApellidos, file.path("eco.wav"))
Aviso:
In writeWave(EcoNombreApellidos, file.path("eco.wav")) :
channels' data will be rounded to integers for writing the
> alreves <- rev(NomApe, output="Wave")
> writeWave(alreves, file.path("alreves.wav"))
> NombrePrimerApellido <- cutw(NombreApellidos, from=0.0,to=0.5)
> writeWave(NombrePrimerApellido, file.path("nombreyprimer
> NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.5)
> SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35)
> NombreYSegundoApellido <- pastew(SegundoApellido, NombreSi
> writeWave(NombreYSegundoApellido, file.path("nombreysegun

```

```

# guardamos el sonido de nuestro nombre y apellidos al revés en disco
writeWave(alreves, file.path("alreves.wav"))

# EJERCICIO 1 ADICIONAL
# eliminamos el segundo apellido
NombrePrimerApellido <- cutw(NombreApellidos, from=0.0,to=1, output="Wave")
# guardamos el sonido de nuestro nombre y primer apellido en disco
writeWave(NombreYPrimerApellido, file.path("nombreyprimerapellido.wav"))

# EJERCICIO 2 ADICIONAL
# nos quedamos solo con nuestro nombre
NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.5, output="Wave")
# nos quedamos solo con nuestro segundo apellido
SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35, output="Wave")
# unión del nombre y segundo apellido
NombreYSegundoApellido <- pastew(SegundoApellido, NombreSinApellidos, output="
# guardamos el sonido de nuestro nombre y segundo apellido en disco
writeWave(NombreYSegundoApellido, file.path("nombreysegundoapellido.wav"))

```

Nombre	Fecha de modificación	Tipo	Tamaño
.Rhstory	24/05/2025 10:20	Archivo de origen ...	2 KB
alreves	24/05/2025 10:21	Archivo WAV	129 KB
apellidos	22/05/2025 20:50	Archivo WAV	83 KB
basico	24/05/2025 10:21	Archivo WAV	129 KB
basicoConFiltro15	23/05/2025 22:02	Archivo WAV	129 KB
basicoConFiltro35	23/05/2025 22:02	Archivo WAV	129 KB
basicoConSilencio	23/05/2025 22:01	Archivo WAV	223 KB
deletePrimerApellido	23/05/2025 23:28	Archivo WAV	82 KB
eco	24/05/2025 10:21	Archivo WAV	437 KB
filtrado	24/05/2025 10:21	Archivo WAV	129 KB
nombre	22/05/2025 20:49	Archivo WAV	47 KB
nombreCambioPrimerSegundoApellido	23/05/2025 22:53	Archivo WAV	129 KB
P5-sonido	24/05/2025 10:20	Archivo de origen R	6 KB
P5-sonido.RData	24/05/2025 10:20	R Workspace	2.339 KB
nombreyprimerapellido	24/05/2025 10:22	Archivo WAV	94 KB
nombreysegundoapellido	24/05/2025 10:25	Archivo WAV	80 KB

c. Información y ondas del nombre con solo el primer apellido y con solo el segundo apellido (ejercicio 3).

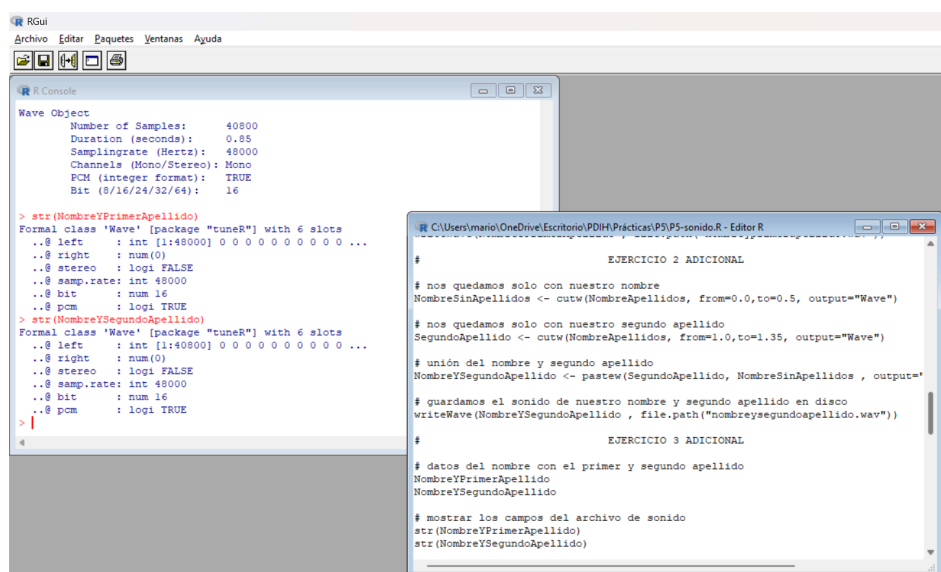
En este ejercicio vamos a ver la información respectiva de los sonidos que contienen nuestro nombre y primer apellido y el de nuestro nombre y segundo apellido. Lo primero de todo, mostraremos los datos de cada uno de los sonidos.

```
NombreYPrimerApellido  
NombreYSegundoApellido
```

```
> NombreYPrimerApellido  
  
Wave Object  
  Number of Samples: 48000  
  Duration (seconds): 1  
  Samplingrate (Hertz): 48000  
  Channels (Mono/Stereo): Mono  
  PCM (integer format): TRUE  
  Bit (8/16/24/32/64): 16  
  
> NombreYSegundoApellido  
  
Wave Object  
  Number of Samples: 40800  
  Duration (seconds): 0.85  
  Samplingrate (Hertz): 48000  
  Channels (Mono/Stereo): Mono  
  PCM (integer format): TRUE  
  Bit (8/16/24/32/64): 16
```

Ahora, pasamos a obtener la información de las cabeceras de ambos sonidos con el método `str`.

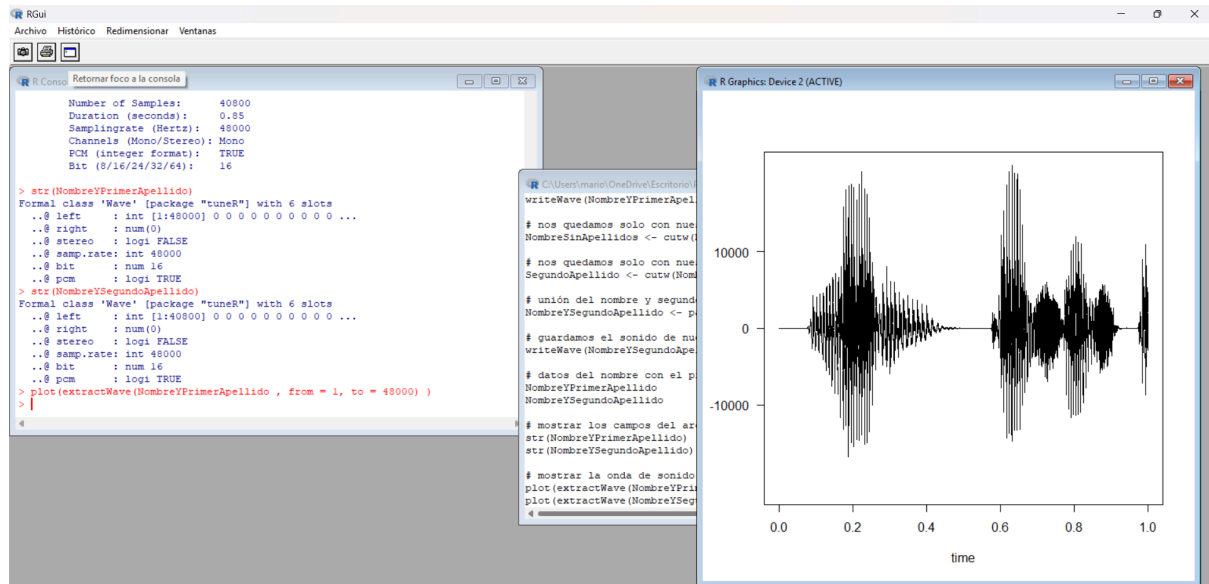
```
str(NombreYPrimerApellido)  
str(NombreYSegundoApellido)
```



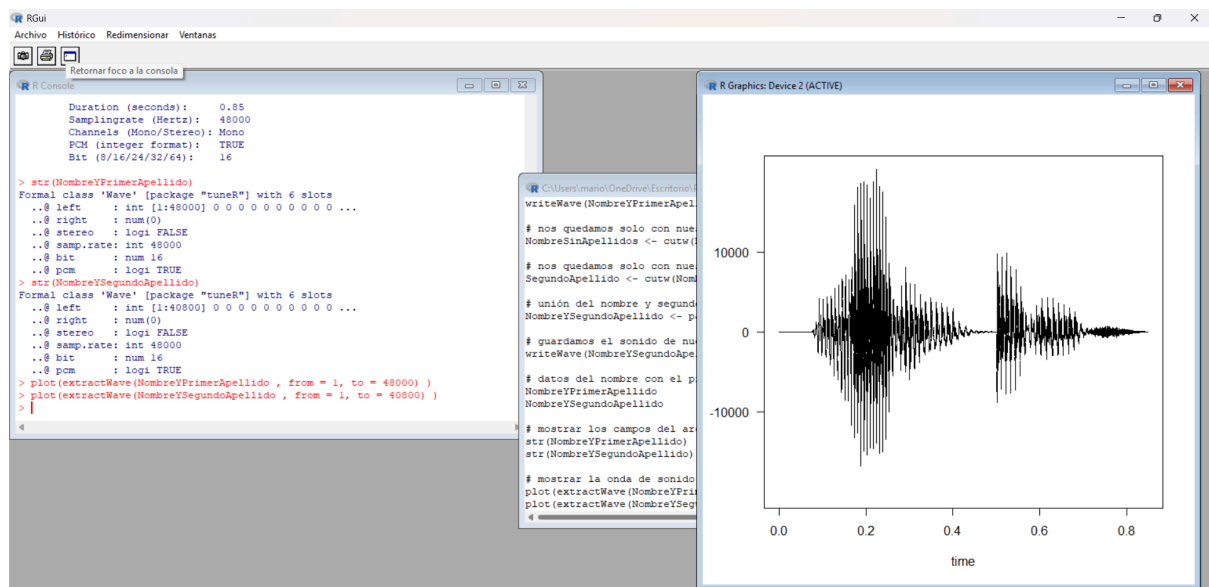
```
RGui  
Archivo Editar Paquetes Ventanas Ayuda  
  
R Console  
Wave Object  
  Number of Samples: 40800  
  Duration (seconds): 0.85  
  Samplingrate (Hertz): 48000  
  Channels (Mono/Stereo): Mono  
  PCM (integer format): TRUE  
  Bit (8/16/24/32/64): 16  
  
> str(NombreYPrimerApellido)  
Formal class 'Wave' [package 'tuneR'] with 6 slots  
..$ left : int [1:48000] 0 0 0 0 0 0 0 0 ...  
..$ right : num(0)  
..$ stereo : logi FALSE  
..$ samp.rate: int 48000  
..$ bit : num 16  
..$ pcm : logi TRUE  
  
> str(NombreYSegundoApellido)  
Formal class 'Wave' [package 'tuneR'] with 6 slots  
..$ left : int [1:40800] 0 0 0 0 0 0 0 0 ...  
..$ right : num(0)  
..$ stereo : logi FALSE  
..$ samp.rate: int 48000  
..$ bit : num 16  
..$ pcm : logi TRUE  
  
# EJERCICIO 2 ADICIONAL  
# nos quedamos solo con nuestro nombre  
NombreSinApellidos <- cutw(NombreApellidos, from=0.0,to=0.5, output="Wave")  
# nos quedamos solo con nuestro segundo apellido  
SegundoApellido <- cutw(NombreApellidos, from=1.0,to=1.35, output="Wave")  
# unión del nombre y segundo apellido  
NombreYSegundoApellido <- pasteW(SegundoApellido, NombreSinApellidos, output="")  
# guardamos el sonido de nuestro nombre y segundo apellido en disco  
writeWave(NombreYSegundoApellido, file.path("NombreYsegundoapellido.wav"))  
  
# EJERCICIO 3 ADICIONAL  
# datos del nombre con el primer y segundo apellido  
NombreYPrimerApellido  
NombreYSegundoApellido  
  
# mostrar los campos del archivo de sonido  
str(NombreYPrimerApellido)  
str(NombreYSegundoApellido)
```

Finalmente, mostraremos las ondas de estos sonidos con la función *plot*. Podremos ver, si analizamos las ondas del nombre y de los apellidos creados en el ejercicio 2 mínimo que coincide con la onda del nombre y de los apellidos, cada uno separado (el nombre con el primer apellido y el nombre con el segundo).

- Nombre con el primer apellido



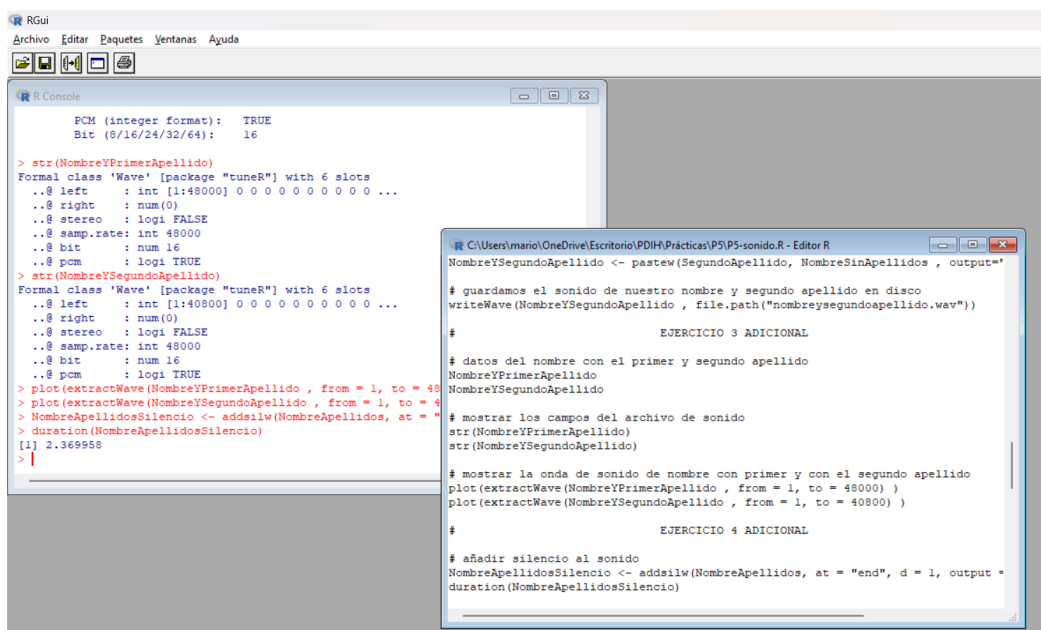
- Nombre con el segundo apellido



d. Añadir silencio al final (ejercicio adicional 4).

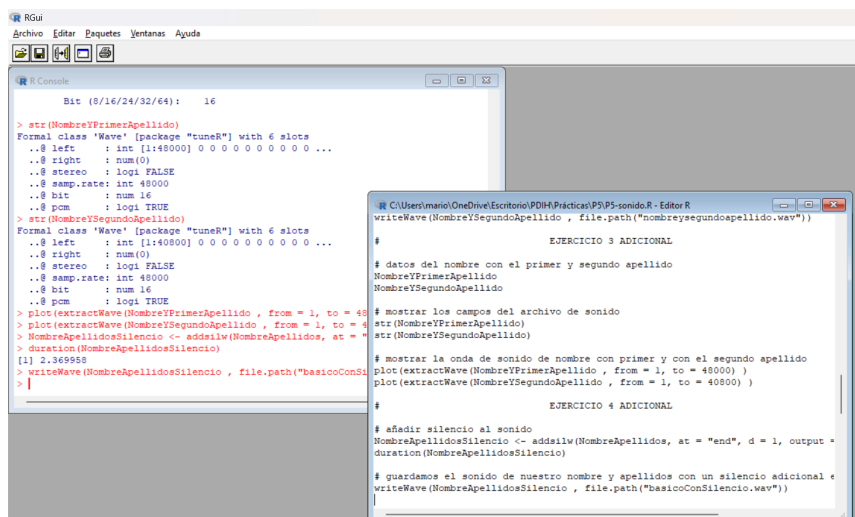
Para este ejercicio vamos a añadir sonido al final del fichero de sonido de nuestro nombre y apellido. Para ello, usaremos la función *addsilw*, donde añadiremos 1 segundo de silencio al final, cuando se digan nuestro nombre y apellidos. Luego, miraremos la duración del fichero de sonido creado con la función *duration*.

```
NombreApellidosSilencio <- addsilw(NombreApellidos, at = "end", d = 1,
output = "Wave")
duration(NombreApellidosSilencio)
```



Finalmente, guardamos en disco el fichero creado como "basicoConSilencio.wav" con la función *writeWave*.

```
writeWave(NombreApellidosSilencio, file.path("basicoConSilencio.wav"))
```



P5

← → ↑ ↺

Escritorio > PDIH > Prácticas > P5

Nuevo

Cortar

Copiar

Pegar

Eliminar

Ordenar

Ver

Inicio

Galería

Escritorio

Downloads

Documentos

Imágenes

Música

Videos

TFG_Pagina

ejemplo

P5

S-sonido

Este equipo

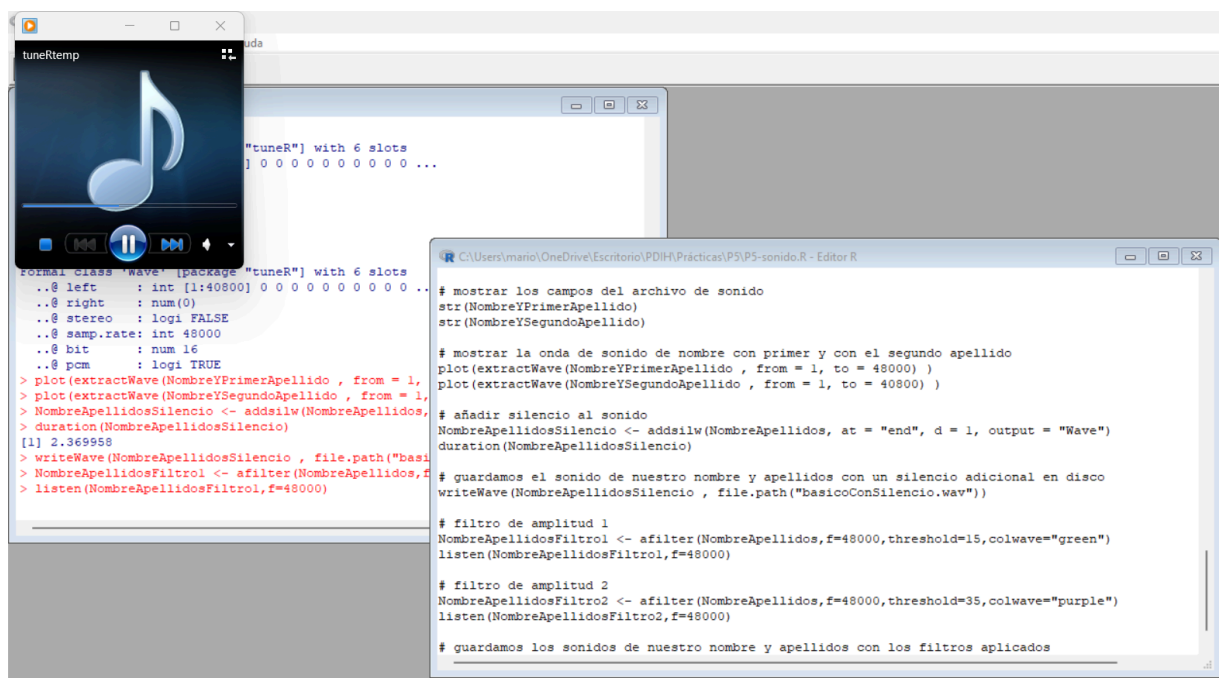
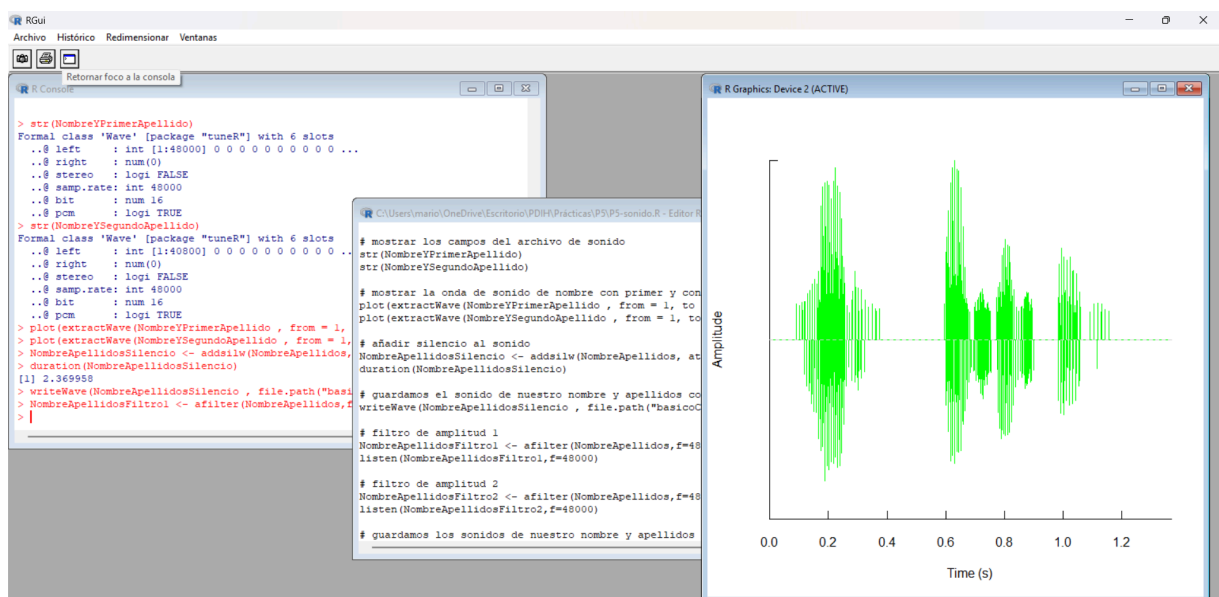
Red

Nombre	Fecha de modificación	Tipo	Tamaño
.Rhstory	24/05/2025 10:20	Archivo de origen ...	2 KB
aireves	24/05/2025 10:21	Archivo WAV	129 KB
apellidos	22/05/2025 20:50	Archivo WAV	83 KB
basico	24/05/2025 10:21	Archivo WAV	129 KB
basicoConFiltro15	23/05/2025 22:02	Archivo WAV	129 KB
basicoConFiltro35	23/05/2025 22:02	Archivo WAV	129 KB
basicoConSilencio	24/05/2025 10:38	Archivo WAV	223 KB
deletePrimerApellido	23/05/2025 23:28	Archivo WAV	82 KB
eco	24/05/2025 10:21	Archivo WAV	437 KB
filtrado	24/05/2025 10:21	Archivo WAV	129 KB
nombre	22/05/2025 20:49	Archivo WAV	47 KB
nombreCambioPrimerSegundoApellido	23/05/2025 22:53	Archivo WAV	129 KB
P5-sonido	24/05/2025 10:20	Archivo de origen R	6 KB
.RData	24/05/2025 10:20	R Workspace	2.339 KB
nombreyprimerapellido	24/05/2025 10:22	Archivo WAV	94 KB
nombreysegundoapellido	24/05/2025 10:25	Archivo WAV	80 KB

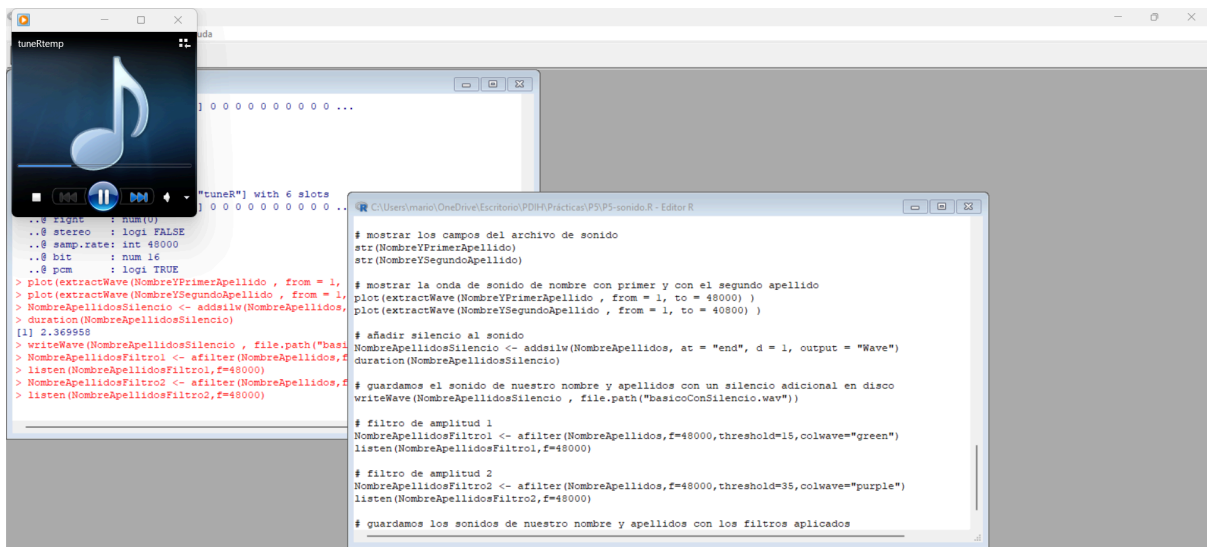
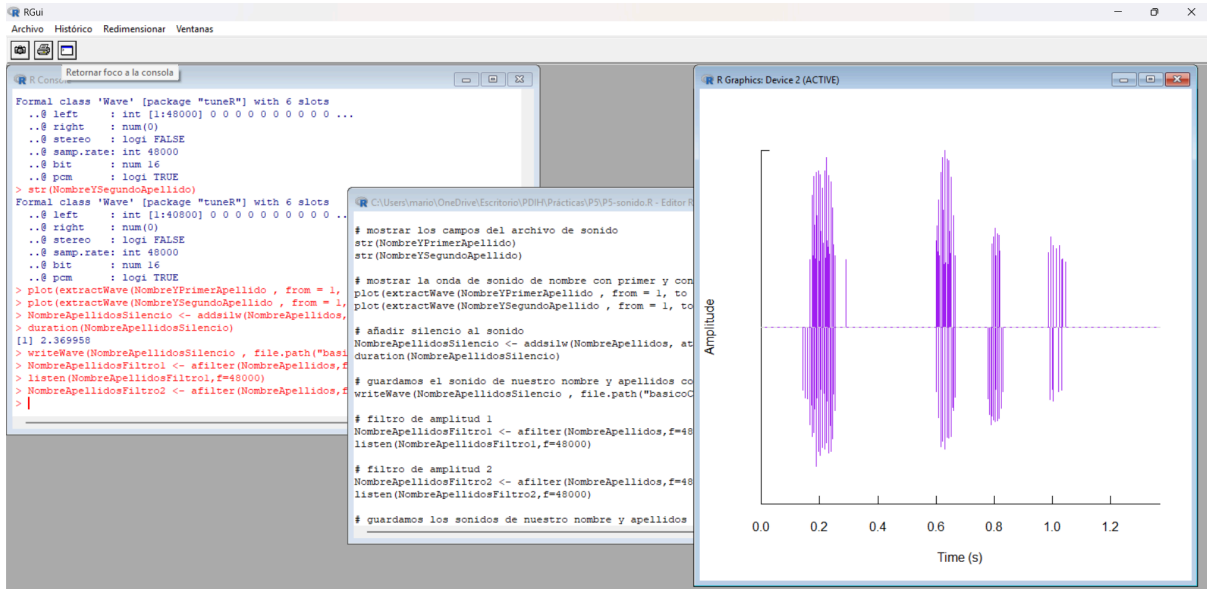
e. Aplicación de filtro de amplitud (ejercicio adicional 5).

Para este ejercicio vamos a coger nuestro fichero de sonido donde se escucha nuestro nombre y apellidos para aplicar un filtro de amplitud, eliminando las muestras que sean inferiores a cierto nivel (descrito con *threshold*), para luego visualizar el sonido resultante y guardarlo en disco. Para ello usaremos la función *afilter* para aplicar el filtro de amplitud y *listen* para visualizarlo. Aplicaremos dos filtros en donde se modificará el número de *threshold*, viendo que, a mayor número (mayor número de muestras eliminadas), peor se escucha el nombre con los apellidos.

```
NombreApellidosFiltro1 <-  
afilter(NombreApellidos,f=48000,threshold=15,colwave="green")  
listen(NombreApellidosFiltro1,f=48000)
```



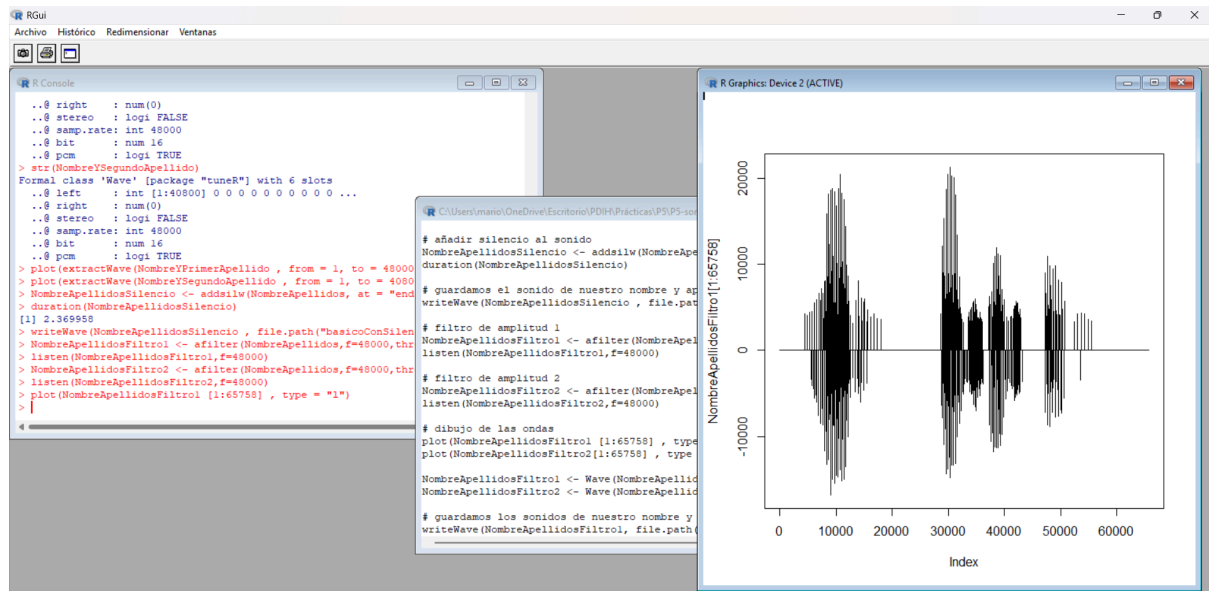
```
NombreApellidosFiltro2 <-
afilter(NombreApellidos,f=48000,threshold=35,colwave="purple")
listen(NombreApellidosFiltro2,f=48000)
```



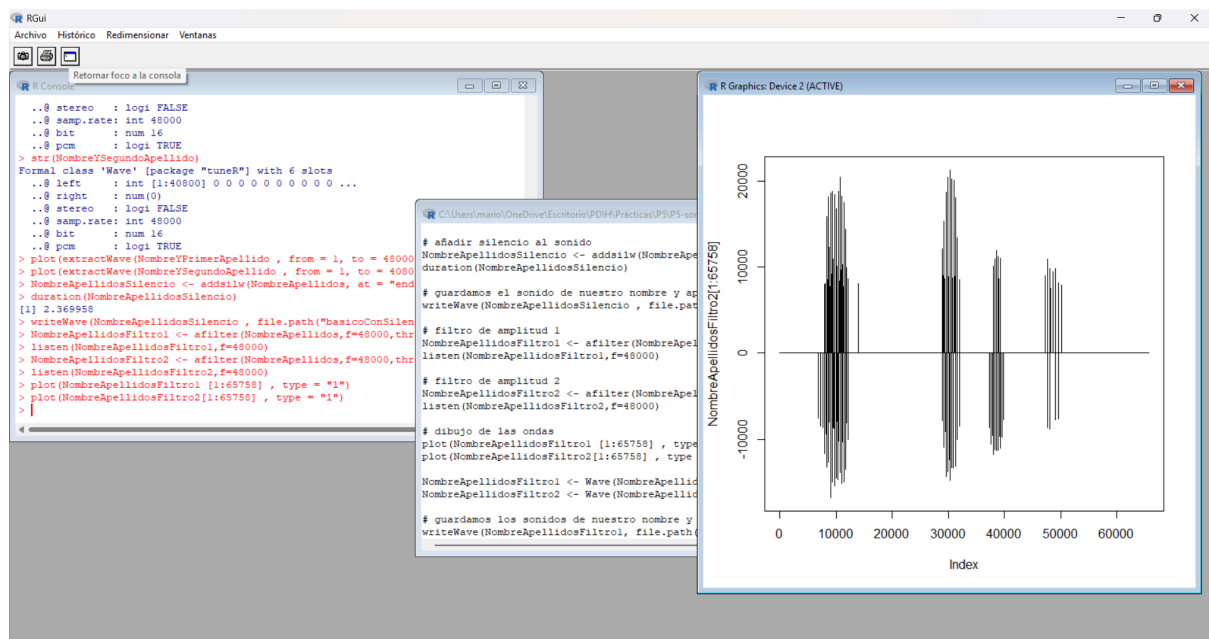
Dado que la función de filtro devuelve arrays con las muestras, para visualizar las ondas debemos acceder al array indicando los elementos a representar.

```
plot(NombreApellidosFiltro1 [1:65758] , type = "l")
plot(NombreApellidosFiltro2[1:65758] , type = "l")
```

- Nombre y apellidos con el primer filtro



- Nombre y apellidos con el segundo filtro



Finalmente guardaremos los dos ficheros de sonido como “basicoConFiltro15.wav” y “basicoConFiltro35.wav” con la función *writeWave*. Como son arrays, debemos de convertirlo primero a onda con la función *Wave*.

```
NombreApellidosFiltro1 <- Wave(NombreApellidosFiltro1)
NombreApellidosFiltro2 <- Wave(NombreApellidosFiltro2)

writeWave(NombreApellidosFiltro1, file.path("basicoConFiltro15.wav"))
writeWave(NombreApellidosFiltro2, file.path("basicoConFiltro35.wav"))
```

```
..@ bit      : num 16
..@ pcm      : logi TRUE
> plot(extractWave(NombreYPrimerApellido , from = 1, to = 48000) )
> plot(extractWave(NombreYSegundoApellido , from = 1, to = 48000) )
> NombreApellidosSilencio <- addsilw(NombreApellidos, at = "end", d = 1, output =
> duration(NombreApellidosSilencio)
[1] 2.369558
> writeWave(NombreApellidosSilencio , file.path("basicoConSilencio.wav"))
> NombreApellidosFiltro1 <- afilter(NombreApellidos,f=48000,thr
> listen(NombreApellidosFiltro1,f=48000)
> NombreApellidosFiltro2 <- afilter(NombreApellidos,f=48000,thr
> listen(NombreApellidosFiltro2,f=48000)
> plot(NombreApellidosFiltro1 [1:65758] , type = "l")
> plot(NombreApellidosFiltro2[1:65758] , type = "l")
> NombreApellidosFiltro1 <- Wave(NombreApellidosFiltro1)
> NombreApellidosFiltro2 <- Wave(NombreApellidosFiltro2)
Avisos:
1: In .local(left, ...) : 'samp.rate' not specified, assuming 4
2: In .local(left, ...) : 'bit' not specified, assuming 16bit
> NombreApellidosFiltro1 <- Wave(NombreApellidosFiltro1)
Avisos:
1: In .local(left, ...) : 'samp.rate' not specified, assuming 4
2: In .local(left, ...) : 'bit' not specified, assuming 16bit
> writeWave(NombreApellidosFiltro1, file.path("basicoConFiltro15.wav"))
> writeWave(NombreApellidosFiltro2, file.path("basicoConFiltro35.wav"))
> |

# Añadir silencio al sonido
NombreApellidosSilencio <- addsilw(NombreApellidos, at = "end", d = 1, output =
duration(NombreApellidosSilencio)

# guardamos el sonido de nuestro nombre y apellidos con un silencio adicional e
writeWave(NombreApellidosSilencio , file.path("basicoConSilencio.wav"))

# filtro de amplitud 1
NombreApellidosFiltro1 <- afilter(NombreApellidos,f=48000,threshold=15,colwave=
listen(NombreApellidosFiltro1,f=48000)

# filtro de amplitud 2
NombreApellidosFiltro2 <- afilter(NombreApellidos,f=48000,threshold=35,colwave=
listen(NombreApellidosFiltro2,f=48000)

# dibujo de las ondas
plot(NombreApellidosFiltro1 [1:65758] , type = "l")
plot(NombreApellidosFiltro2[1:65758] , type = "l")

NombreApellidosFiltro1 <- Wave(NombreApellidosFiltro1)
NombreApellidosFiltro2 <- Wave(NombreApellidosFiltro2)

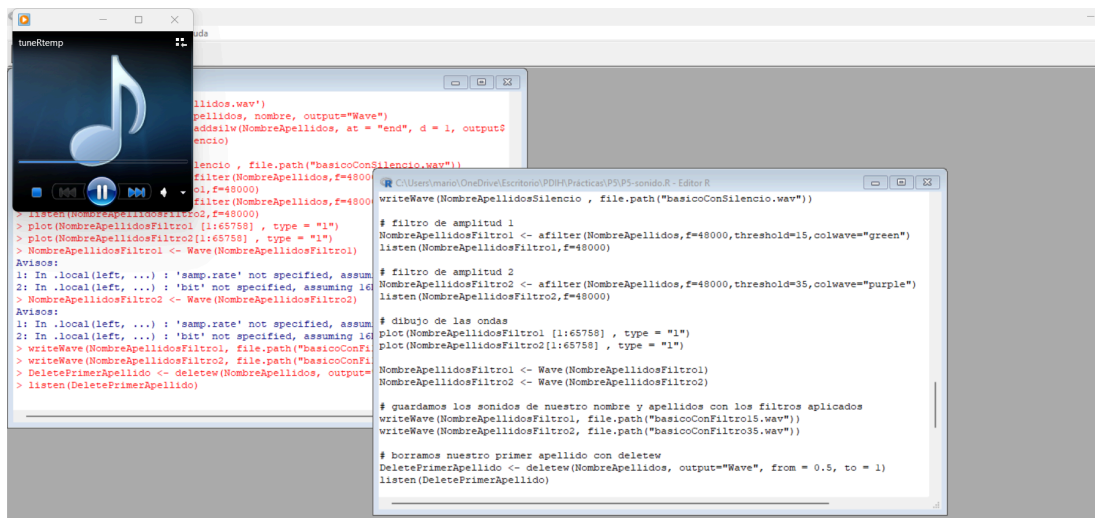
# guardamos los sonidos de nuestro nombre y apellidos con los filtros aplicados
writeWave(NombreApellidosFiltro1, file.path("basicoConFiltro15.wav"))
writeWave(NombreApellidosFiltro2, file.path("basicoConFiltro35.wav"))
```

Nombre	Fecha de modificación	Tipo	Tamaño
.Rhistory	23/05/2025 16:28	Archivo de origen...	3 KB
alreves	23/05/2025 14:24	Archivo WAV	129 KB
apellidos	22/05/2025 20:50	Archivo WAV	83 KB
basico	23/05/2025 14:23	Archivo WAV	129 KB
basicoConSilencio	23/05/2025 16:30	Archivo WAV	223 KB
eco	23/05/2025 14:23	Archivo WAV	437 KB
filtrado	23/05/2025 14:23	Archivo WAV	129 KB
nombre	22/05/2025 20:49	Archivo WAV	47 KB
nombreayprimerapellido	23/05/2025 16:29	Archivo WAV	94 KB
nombreaysegundoapellido	23/05/2025 16:29	Archivo WAV	80 KB
P5-sonido	23/05/2025 16:28	Archivo de origen R	5 KB
basicoConFiltro15	23/05/2025 16:35	Archivo WAV	129 KB
basicoConFiltro35	23/05/2025 16:35	Archivo WAV	129 KB
.RData	23/05/2025 16:28	R Workspace	693 KB

f. Eliminar porción de sonido con deletew (ejercicio adicional 6)

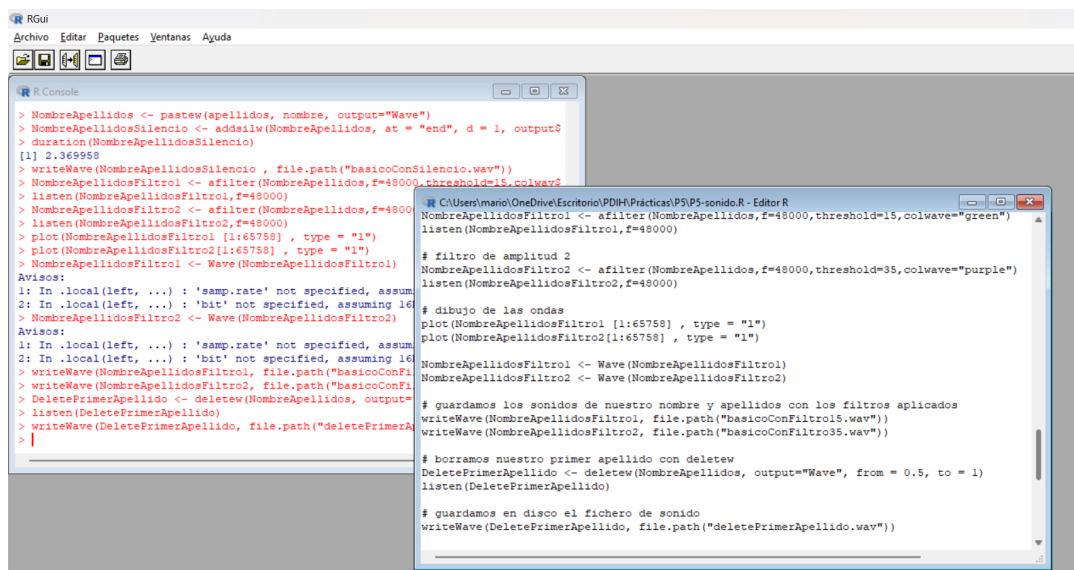
En este ejercicio vamos a eliminar una porción del fichero de sonido con nuestro nombre y apellidos. En el, solo dejaremos el nombre y el segundo apellido. Para ello, usaremos la función *deletew* indicando la parte que queremos eliminar (con *from* y *to*).

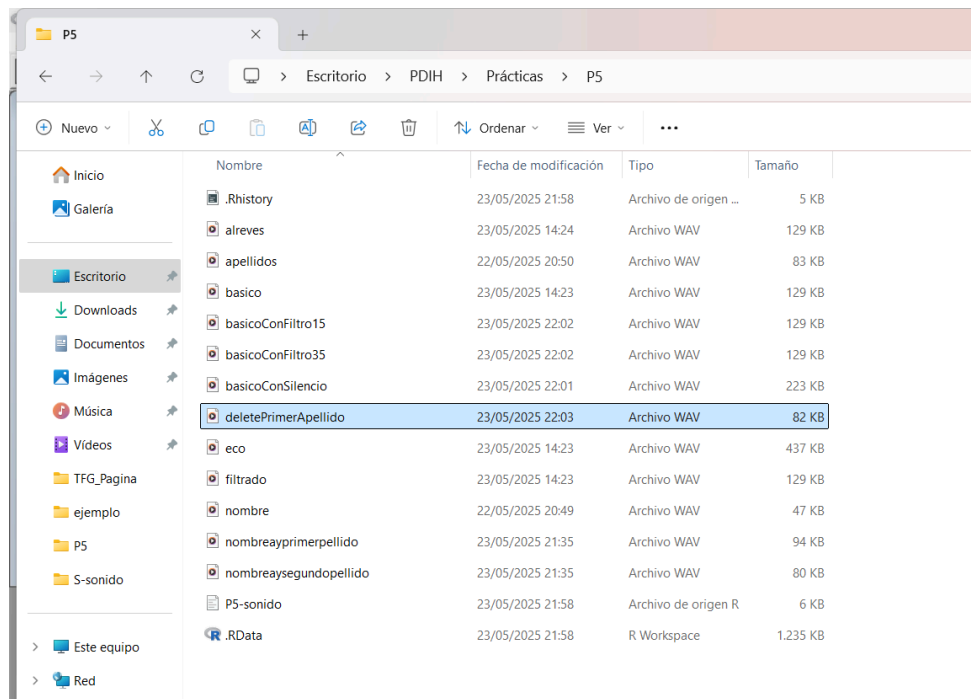
```
DeletePrimerApellido <- deletew(NombreApellidos, output="Wave", from = 0.5,
to = 1)
listen>DeletePrimerApellido)
```



Finalmente, guardamos el fichero en disco con el nombre “deletePrimerApellido.wav”.

```
writeWave>DeletePrimerApellido, file.path("deletePrimerApellido.wav"))
```





g. Cambiar primer apellido por segundo (ejercicio adicional 7)

Este ejercicio consistirá en intercambiar los apellidos con nuestro nombre, saliendo de esta forma en el fichero de sonido el nombre, nuestro segundo apellido y luego nuestro primer apellido (usando solamente la variable que tiene nuestro nombre y apellidos). Para ello, se deberá de usar la función *cutw*, dónde nos quedaremos solamente con los dos apellidos (cada uno en una variable diferente) y, mediante la función *deletew* nos quedamos con la parte donde se indica nuestro nombre, eliminando los apellidos de esta. Finalmente, con la función *pastew*, se insertará a la variable donde esté nuestro nombre el segundo apellido primeramente, y luego nuestro primer apellido.

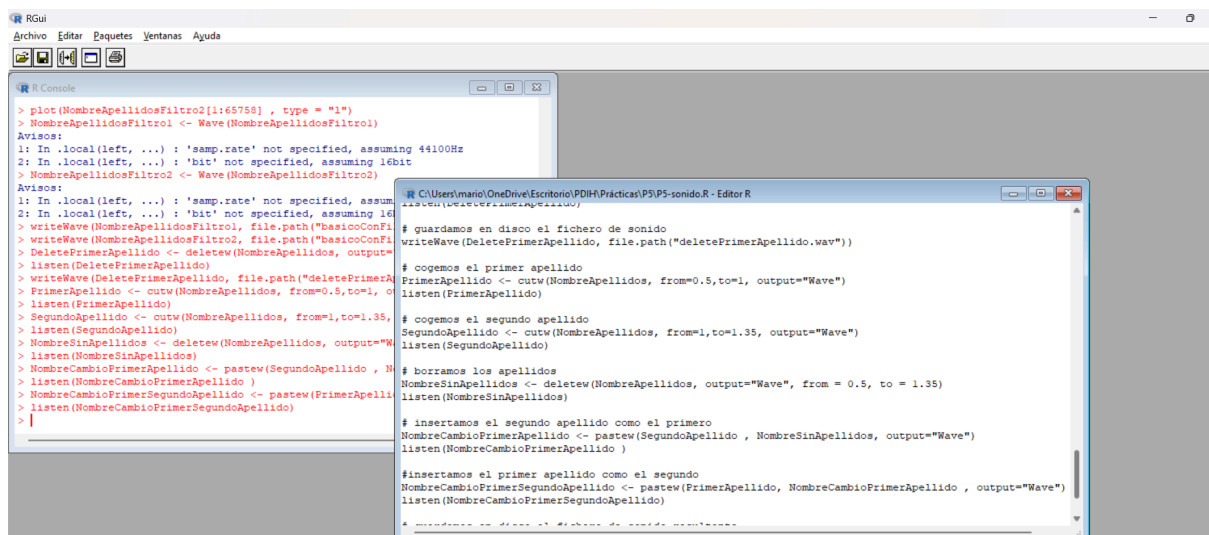
```
PrimerApellido <- cutw(NombreApellidos, from=0.5, to=1, output="Wave")
listen(PrimerApellido)

SegundoApellido <- cutw(NombreApellidos, from=1, to=1.35, output="Wave")
listen(SegundoApellido)

NombreSinApellidos <- deletew(NombreApellidos, output="Wave", from=0.5,
to=1.35)
listen(NombreSinApellidos)

NombreCambioPrimerApellido <- pastew(SegundoApellido, NombreSinApellidos,
output="Wave")
listen(NombreCambioPrimerApellido)

NombreCambioPrimerSegundoApellido <- pastew(PrimerApellido,
NombreCambioPrimerApellido, output="Wave")
listen(NombreCambioPrimerSegundoApellido)
```



Una vez ejecutado todo el procedimiento y ver que el fichero de sonido resultante es el intercambio de apellidos, pasamos a guardarlo en disco en un archivo llamado "nombreCambioPrimerSegundoApellido.wav".

```
writeWave(NombreCambioPrimerSegundoApellido,
file.path("nombreCambioPrimerSegundoApellido.wav"))
```

The screenshot shows the RGui interface. The R Console window displays the execution of R commands, including warnings about sampling rate and bit depth. The script editor window shows the following R code:

```
# C:\Users\mario\OneDrive\Escritorio\PDIH\Prácticas\P5\P5-sonido.R - Editor R
PrimerApellido <- cutw(NombreApellidos, from=0.5, to=1, output="Wave")
listen(PrimerApellido)

# cogemos el segundo apellido
SegundoApellido <- cutw(NombreApellidos, from=1, to=1.35, output="Wave")
listen(SegundoApellido)

# borramos los apellidos
NombreSinApellidos <- deletew(NombreApellidos, output="Wave", from = 0.5, to = 1.35)
listen(NombreSinApellidos)

# insertamos el segundo apellido como el primero
NombreCambioPrimerApellido <- pastew(SegundoApellido, NombreSinApellidos, output="Wave")
listen(NombreCambioPrimerApellido)

# insertamos el primer apellido como el segundo
NombreCambioPrimerSegundoApellido <- pastew(PrimerApellido, NombreCambioPrimerApellido, output="Wave")
listen(NombreCambioPrimerSegundoApellido)

# guardamos en disco el fichero de sonido resultante
writeWave(NombreCambioPrimerSegundoApellido, file.path("nombreCambioPrimerSegundoApellido.wav"))
```

The screenshot shows a Windows File Explorer window with the address bar set to 'Escritorio > PDIH > Prácticas > P5'. The file list is as follows:

Nombre	Fecha de modificación	Tipo	Tamaño
.Rhstory	23/05/2025 21:58	Archivo de origen ...	5 KB
alreves	23/05/2025 14:24	Archivo WAV	129 KB
apellidos	22/05/2025 20:50	Archivo WAV	83 KB
basico	23/05/2025 14:23	Archivo WAV	129 KB
basicoConFiltro15	23/05/2025 22:02	Archivo WAV	129 KB
basicoConFiltro35	23/05/2025 22:02	Archivo WAV	129 KB
basicoConSilencio	23/05/2025 22:01	Archivo WAV	223 KB
deletePrimerApellido	23/05/2025 22:03	Archivo WAV	82 KB
eco	23/05/2025 14:23	Archivo WAV	437 KB
filtrado	23/05/2025 14:23	Archivo WAV	129 KB
nombre	22/05/2025 20:49	Archivo WAV	47 KB
nombreyayprimerapellido	23/05/2025 21:35	Archivo WAV	94 KB
nombreyaysegundoapellido	23/05/2025 21:35	Archivo WAV	80 KB
P5-sonido	23/05/2025 21:58	Archivo de origen R	6 KB
P5-sonido.RData	23/05/2025 21:58	R Workspace	1.235 KB
nombreCambioPrimerSegundoApellido	23/05/2025 22:53	Archivo WAV	129 KB

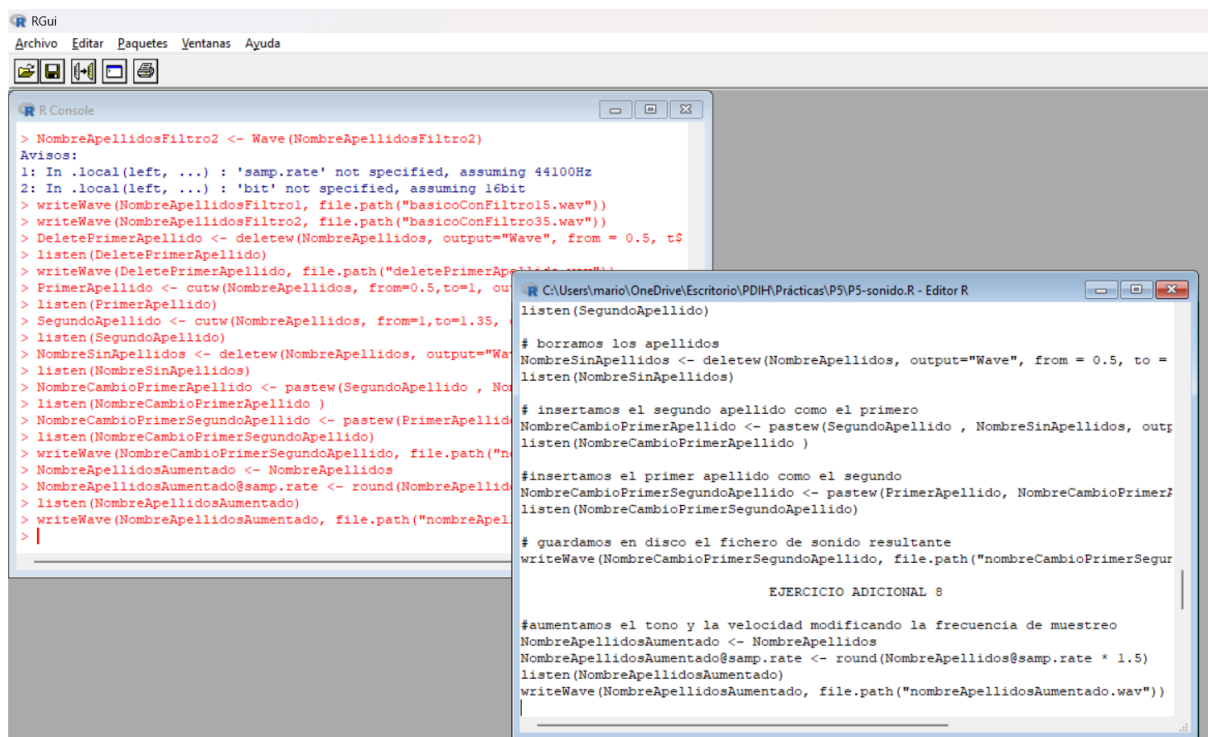
h. Aumentar el tono y la velocidad modificando la frecuencia de muestreo (ejercicio adicional 8)

Para este ejercicio se va a aumentar el tono y la velocidad del nombre y de los apellidos modificando la frecuencia de muestreo. Para ello, se deberá multiplicar la frecuencia de muestreo (en mi caso he puesto 1.5) y redondear (con la función *round*), de tal forma que aumentemos el tono y la velocidad de reproducción mediante el incremento de la frecuencia.

```
NombreApellidosAumentado <- NombreApellidos
NombreApellidosAumentado@samp.rate <- round(NombreApellidos@samp.rate * 1.5)
listen(NombreApellidosAumentado)
```

Una vez ejecutado todo el procedimiento y ver que el fichero de sonido resultante es el aumento del tono y velocidad del nombre y apellidos, pasamos a guardarlo en disco en un archivo llamado "nombreApellidosAumentado.wav".

```
writeWave(NombreApellidosAumentado,
file.path("nombreApellidosAumentado.wav"))
```



P5

<

>

↑

↺

🖨

>

Escritorio

>

PDIH

>

Prácticas

>

P5

+ Nuevo

✂

📄

📁

🔍

🔗

🗑

⬆ Ordenar

☰ Ver

⋮

	Nombre	Fecha de modificación	Tipo	Tamaño
Inicio	.RData	29/05/2025 21:24	R Workspace	2.864 KB
Galería	.Rhistory	29/05/2025 21:24	Archivo de origen ...	12 KB
Escritorio	alreves	29/05/2025 21:26	Archivo WAV	129 KB
Downloads	apellidos	22/05/2025 20:50	Archivo WAV	83 KB
Documentos	basico	24/05/2025 10:21	Archivo WAV	129 KB
Imágenes	basicoConFiltro15	29/05/2025 21:27	Archivo WAV	129 KB
Música	basicoConFiltro35	29/05/2025 21:27	Archivo WAV	129 KB
Videos	basicoConSilencio	29/05/2025 21:26	Archivo WAV	223 KB
TFG_Pagina	deletePrimerApellido	29/05/2025 21:27	Archivo WAV	82 KB
HARRY POTTER	eco	29/05/2025 21:25	Archivo WAV	437 KB
Temporada1	filtrado	29/05/2025 21:25	Archivo WAV	129 KB
P5	nombre	22/05/2025 20:49	Archivo WAV	47 KB
	nombreApellidosAumentado	29/05/2025 21:39	Archivo WAV	129 KB
	nombreCambioPrimerSegundoApellido	29/05/2025 21:28	Archivo WAV	129 KB
	nombreyprimerapellido	29/05/2025 21:26	Archivo WAV	94 KB
	nombreysegundoapellido	29/05/2025 21:26	Archivo WAV	80 KB
	P5-sonido	29/05/2025 21:24	Archivo de origen R	8 KB

> Este equipo

> Red

i. Reordenación de segmentos (ejercicio adicional 9)

Para este ejercicio vamos a coger la variable que tiene el nombre y apellidos y vamos a mezclar aleatoriamente los sonidos de esta. Para ello, cortaremos el sonido en segmentos para posteriormente reordenarlos.

Lo primero de todo es definir un segmento de cierta duración (en mi caso he puesto 0.3 segundos) y calcular el número de muestras basándose en la frecuencia de muestreo del objeto *NombreYApellidos* (usaremos las funciones *round* y *floor* para ello, donde esta última redondea hacia abajo). Luego, vamos extrayendo cada segmento del sonido original para luego reordenar aleatoriamente (esto se hace con la función *sample*), combinándolo en un solo objeto de audio usando la función *pastew*.

```
tSeg <- 0.3
tamSeg <- round(tSeg * NombreApellidos@samp.rate)
numSeg <- floor(length(NombreApellidos@left) / tamSeg)

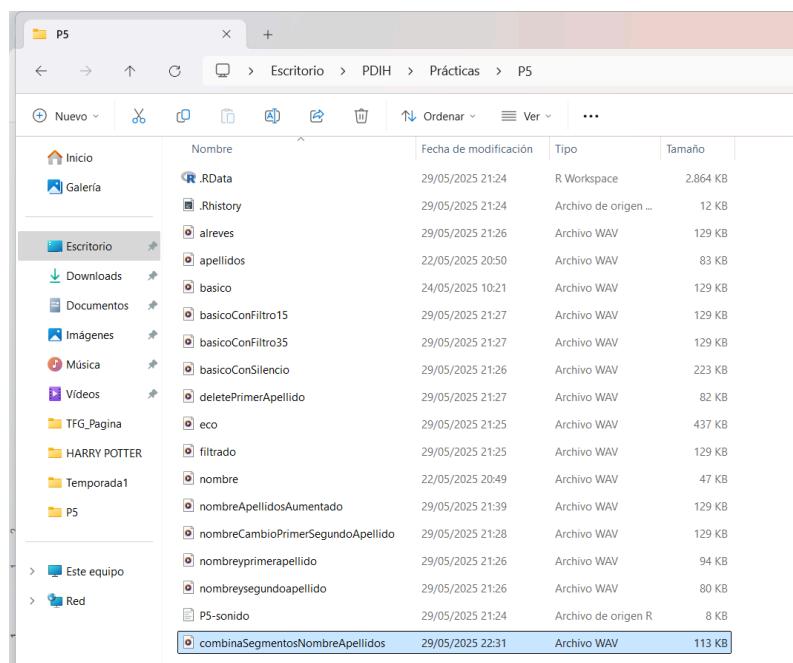
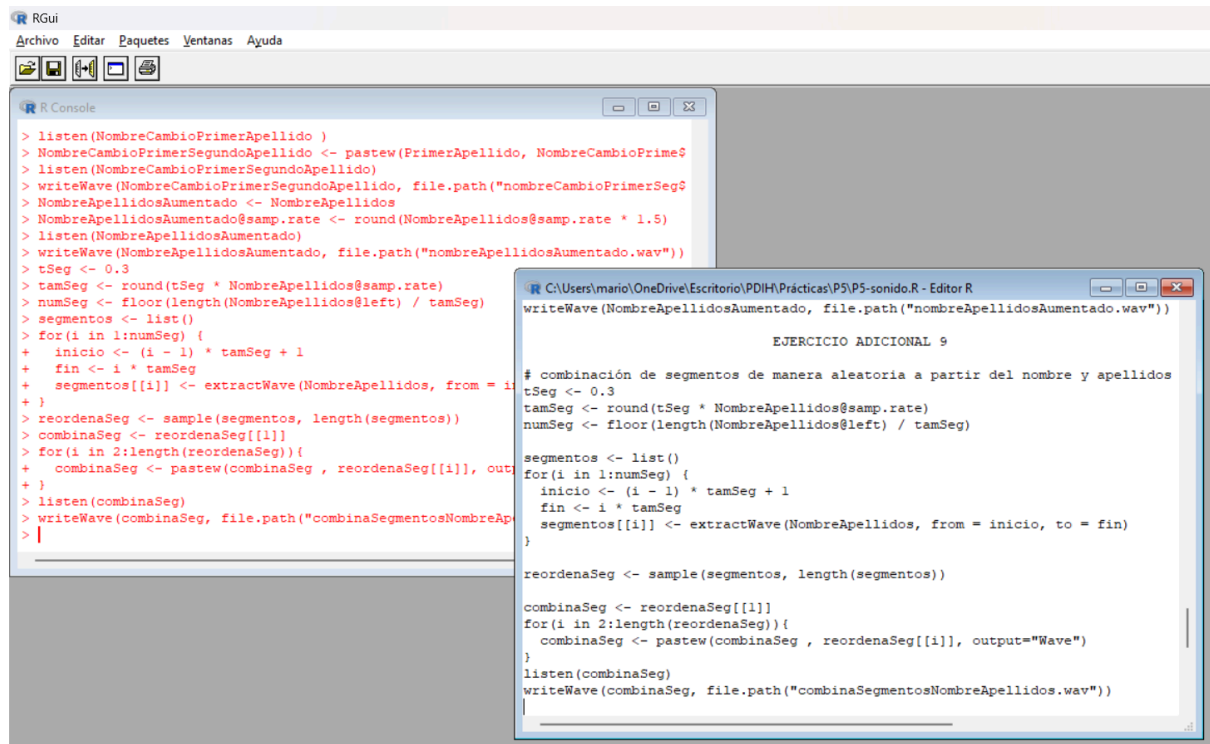
segmentos <- list()
for(i in 1:numSeg) {
  inicio <- (i - 1) * tamSeg + 1
  fin <- i * tamSeg
  segmentos[[i]] <- extractWave(NombreApellidos, from = inicio, to = fin)
}

reordenaSeg <- sample(segmentos, length(segmentos))

combinaSeg <- reordenaSeg[[1]]
for(i in 2:length(reordenaSeg)){
  combinaSeg <- pastew(combinaSeg , reordenaSeg[[i]], output="Wave")
}
listen(combinaSeg)
```

Una vez ejecutado todo el procedimiento y ver que el fichero de sonido resultante es la combinación aleatoria de segmentos del nombre y apellidos, pasamos a guardarlo en disco en un archivo llamado "combinaSegmentosNombreApellidos.wav".

```
writeWave(combinaSeg, file.path("combinaSegmentosNombreApellidos.wav"))
```



j. Efecto de eco con retraso atenuado (ejercicio adicional 10)

Para este ejercicio se va a aplicar un efecto de eco donde se suma el sonido del nombre y apellidos con una versión retrasada y atenuada. Para ello, se define el retardo y factor de atenuación que queramos ponerle (en mi caso he puesto 0.2 segundos y 0.6 segundos, respectivamente), así como las muestras de retardo que se tendrán. Luego, se crea un vector que empieza con una cantidad de ceros iguales al número de muestras de retardo, seguido por los datos del sonido original pero teniendo en cuenta el desplazamiento. Una vez hecho esto se suma el vector original con la versión retrasada y atenuada, por lo que se tendrá un audio en el que el eco se superpone a nuestro nombre y apellidos, creando un efecto de repetición rápida.

```
# Efecto con retraso atenuado (eco muy seguido)
retardo <- 0.2
muestrasRetardo <- round(retardo * NombreApellidos@samp.rate)
factorAtenuacion <- 0.6

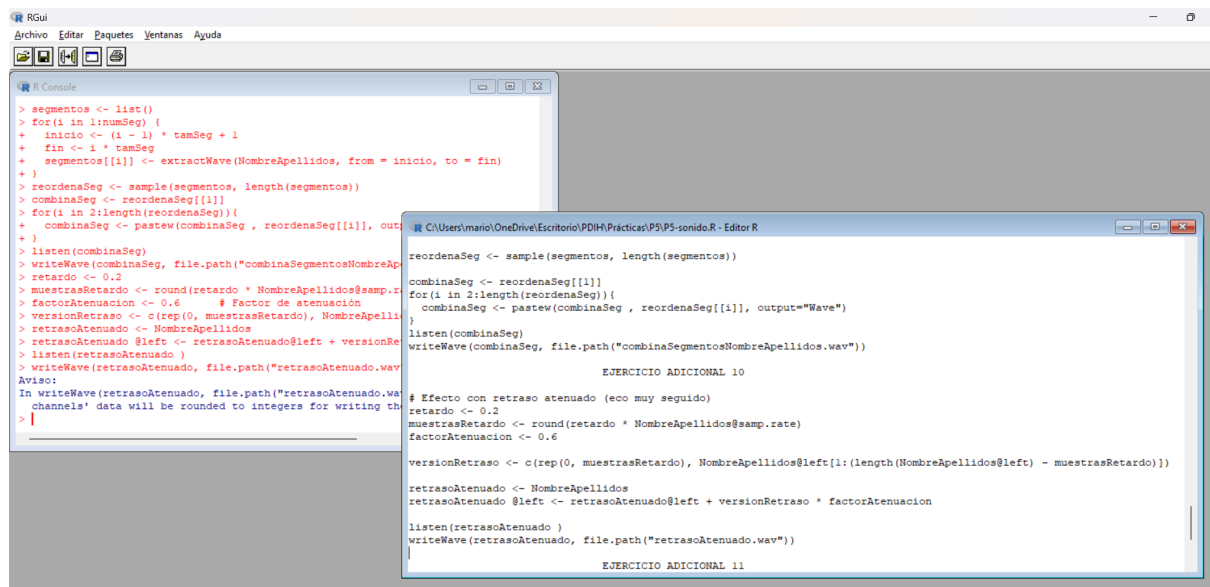
versionRetraso <- c(rep(0, muestrasRetardo),
NombreApellidos@left[1:(length(NombreApellidos@left) - muestrasRetardo)])

retrasoAtenuado <- NombreApellidos
retrasoAtenuado @left <- retrasoAtenuado@left + versionRetraso *
factorAtenuacion

listen(retrasoAtenuado )
```

Una vez ejecutado todo el procedimiento y ver que el fichero de sonido resultante es la unión del eco creado gracias al retraso atenuado y nuestro nombre y apellidos, pasamos a guardarlo en disco en un archivo llamado "retrasoAtenuado.wav".

```
writeWave(retrasoAtenuado, file.path("retrasoAtenuado.wav"))
```



File Explorer window showing the contents of the P5 folder.

Nombre	Fecha de modificación	Tipo	Tamaño
.RData	29/05/2025 21:24	R Workspace	2.864 KB
.Rhistory	29/05/2025 21:24	Archivo de origen ...	12 KB
alreves	29/05/2025 21:26	Archivo WAV	129 KB
apellidos	22/05/2025 20:50	Archivo WAV	83 KB
basico	24/05/2025 10:21	Archivo WAV	129 KB
basicoConFiltro15	29/05/2025 21:27	Archivo WAV	129 KB
basicoConFiltro35	29/05/2025 21:27	Archivo WAV	129 KB
basicoConSilencio	29/05/2025 21:26	Archivo WAV	223 KB
combinaSegmentosNombreApellidos	29/05/2025 22:31	Archivo WAV	113 KB
deletePrimerApellido	29/05/2025 21:27	Archivo WAV	82 KB
eco	29/05/2025 21:25	Archivo WAV	437 KB
filtrado	29/05/2025 21:25	Archivo WAV	129 KB
nombre	22/05/2025 20:49	Archivo WAV	47 KB
nombreApellidosAumentado	29/05/2025 21:39	Archivo WAV	129 KB
nombreCambioPrimerSegundoApellido	29/05/2025 21:28	Archivo WAV	129 KB
nombrePrimerApellido	29/05/2025 21:26	Archivo WAV	94 KB
nombreSegundoApellido	29/05/2025 21:26	Archivo WAV	80 KB
P5-sonido	29/05/2025 21:24	Archivo de origen R	8 KB
retrasoAtenuado	29/05/2025 22:42	Archivo WAV	129 KB

k. Efecto de chorus (ejercicio adicional 11)

Para este ejercicio se va a implementar un efecto de *chorus*, que consiste en mezclar el sonido del nombre y apellidos con una versión retrasada y atenuada de la misma. Para ello, se calcula el retardo en muestras donde se define un retardo (en mi caso he puesto 0.03 segundos), convertido a muestras multiplicando dicho valor por la frecuencia de muestreo del audio (se debe de asegurar que el resultado sea un entero dado que el número de muestras debe de ser exacto, por eso se usa la función *round*). Luego, se hace una copia del objeto del nombre y apellidos asignándolo a la variable *chorus* y, en esta, se crea el efecto, donde se modifica el canal izquierdo de dicho objeto y se le suma una versión retrasada del propio canal.

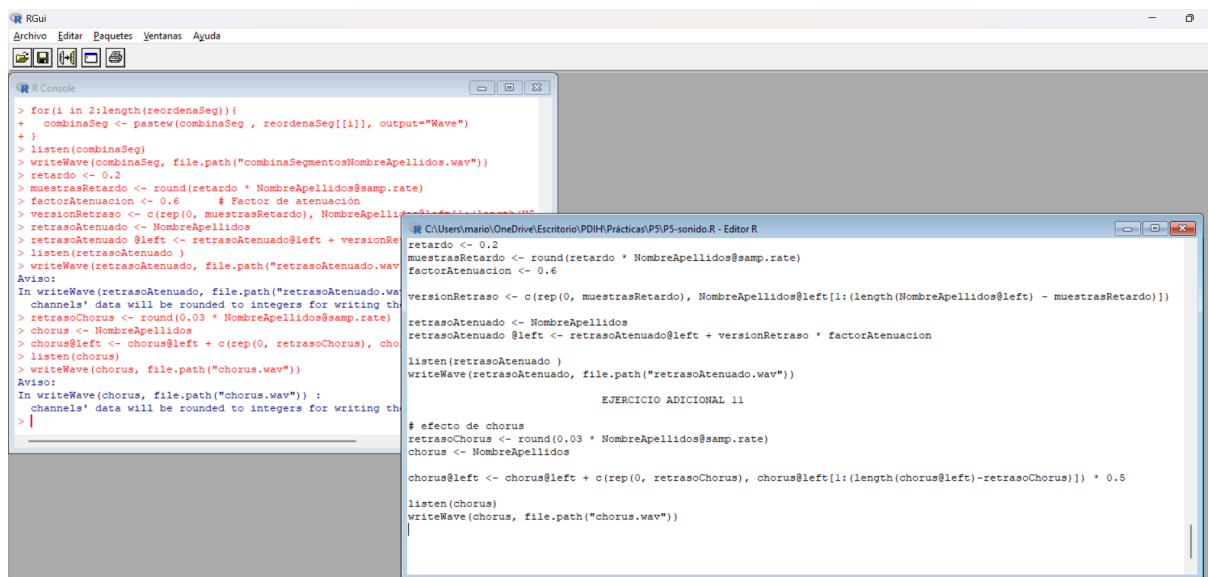
```
retrasoChorus <- round(0.03 * NombreApellidos@samp.rate)
chorus <- NombreApellidos

chorus@left <- chorus@left + c(rep(0, retrasoChorus),
chorus@left[1:(length(chorus@left)-retrasoChorus)]) * 0.5

listen(chorus)
```

Una vez ejecutado todo el procedimiento y ver que el fichero de sonido resultante es el efecto *chorus* deseado (escuchándolo, parece que el sonido creado lo está diciendo un robot), pasamos a guardarlo en disco en un archivo llamado "chorus.wav".

```
writeWave(chorus, file.path("chorus.wav"))
```



Windows Explorer interface showing the file system structure:

- Navigation pane (left):** Inicio, Galería, Escritorio (selected), Downloads, Documentos, Imágenes, Música, Videos, TFG_Pagina, HARRY POTTER, Temporada1, PS.
- Breadcrumb path:** > Escritorio > PDIH > Prácticas > P5
- File list table:**

Nombre	Fecha de modificación	Tipo	Tamaño
RData	29/05/2025 21:24	R Workspace	2.864 KB
Rhistory	29/05/2025 21:24	Archivo de origen ...	12 KB
alreaves	29/05/2025 21:26	Archivo WAV	129 KB
apellidos	22/05/2025 20:50	Archivo WAV	83 KB
basico	24/05/2025 10:21	Archivo WAV	129 KB
basicoConFiltro15	29/05/2025 21:27	Archivo WAV	129 KB
basicoConFiltro35	29/05/2025 21:27	Archivo WAV	129 KB
basicoConSilencio	29/05/2025 21:26	Archivo WAV	223 KB
combinaSegmentosNombreApellidos	29/05/2025 22:31	Archivo WAV	113 KB
deletePrimerApellido	29/05/2025 21:27	Archivo WAV	82 KB
eco	29/05/2025 21:25	Archivo WAV	437 KB
filtrado	29/05/2025 21:25	Archivo WAV	129 KB
nombre	22/05/2025 20:49	Archivo WAV	47 KB
nombreApellidosAumentado	29/05/2025 21:39	Archivo WAV	129 KB
nombreCambioPrimerSegundoApellido	29/05/2025 21:28	Archivo WAV	129 KB
nombreyprimeraapellido	29/05/2025 21:26	Archivo WAV	94 KB
nombreysegundoaapellido	29/05/2025 21:26	Archivo WAV	80 KB
PS-sonido	29/05/2025 21:24	Archivo de origen R	8 KB
retrosoAtenuado	29/05/2025 22:42	Archivo WAV	129 KB
chorus	29/05/2025 22:59	Archivo WAV	129 KB

10. Bibliografía

<https://swad.ugr.es/swad/tmp/33/JTcDKPWV0u0zsLBb4ernVtkrcdemxOPeU7-MJIOF8/P5-Sonido.pdf>

<https://www.tubesoundbcn.com/pedales-efectos-chorus-199?srsId=AfmBOorYxL59iXxhr7PJ4GEa9NckXGq6529R8Q3u01h-IAILER3f67ib>